scientific reports

Check for updates

OPEN

Topology-aware self-supervised multivariate KPI anomaly detection and root cause node localization for network operations

Debiao Luo[1], Weijie Wang[1]✉, Xinyue Liu[1], Wen Yang[1], Ke Hu[1] & Jia Zhang[2]

Multivariate key performance indicator (KPI) streams collected in network operations contain rich evidence for anomaly diagnosis, but practical deployment remains difficult because fault labels are scarce, KPI dependencies are strong across time and metrics, and fault effects propagate through network topology. This paper proposes a topology-aware self-supervised framework for multivariate KPI anomaly detection and root cause node localization in network operations. The framework learns normal behavior from predominantly unlabeled KPI windows by combining masked reconstruction, one-step prediction, and topology-regularized representation learning. It then produces KPI-level, node-level, and system-level anomaly scores, and ranks candidate root cause nodes by combining normalized local anomaly evidence with consistency under a directed multi-hop propagation prior. To improve transparency, the framework also reports residual-based KPI contribution scores and a compact topology-level explanation subgraph. These residual shares are descriptive diagnostic evidence rather than causal or model-gradient feature attributions and do not constitute fault-type classification. Experiments on a Mininet-based testbed with three topology scales and six fault categories yield a five-seed F1-score of 0.9235 ± 0.0037 and an A@1 of 0.7613 ± 0.0163. The mean F1-score is 0.43 percentage points higher than that of DCdetector, although the paired difference is not statistically significant. Under the controlled KPI-plus-topology observability budget, A@1 is 5.47 percentage points higher than that of the restricted-input Nezha adaptation (Holm-adjusted $\mathbf{p = 0.013}$); this comparison does not represent Nezha operating with its native multimodal inputs. External validation on the public Train-Ticket benchmark yields an F1-score of 0.846 and an A@1 of 0.742. Ablation results show that directed propagation improves A@1 by 1.6 percentage points over static undirected proximity, while five-seed results indicate stable optimization. On an NVIDIA RTX 3090, the complete online detection and RCA path requires 12.4 ms in the 48-node setting.

Keywords Network operations, Multivariate KPI anomaly detection, Root cause node localization, Self-supervised learning, Topology priors, Explainable AIOps

Modern network operation environments continuously generate massive volumes of multivariate key performance indicator (KPI) data, including throughput, latency, packet loss, CPU utilization, memory usage, queue length, and interface statistics. These monitoring signals provide the basis for identifying abnormal system behaviors and maintaining service reliability. However, as network infrastructures become increasingly large-scale, dynamic, and software-defined, operation and maintenance tasks are no longer limited to detecting whether an anomaly exists; they also require rapidly identifying *where* the anomaly originates and *why* it propagates across interconnected components. Therefore, anomaly detection and root cause node localization have become two tightly coupled problems in intelligent network operations and AIOps.

Despite substantial progress, three fundamental challenges remain. First, high-quality fault labels are scarce in real-world network operation scenarios, because many failures are rare, complex, and difficult to annotate manually. This makes purely supervised diagnosis approaches difficult to deploy in practice. Second, network KPI streams are strongly correlated across both temporal and spatial dimensions. An abnormal event often manifests as coupled deviations across multiple metrics rather than as an isolated change in a single variable. Traditional threshold-based or univariate statistical approaches are therefore prone to false alarms and missed detections.

[1]Information Network Center, Chengdu University, Chengdu 610106, China. [2]Library, Chengdu University, Chengdu 610106, China. ✉email: wangweijie@cdu.edu.cn

Third, abnormal behaviors in network systems are rarely independent. Instead, they usually propagate through physical links, routing dependencies, traffic paths, or resource-sharing relationships. As a result, accurate anomaly detection alone is insufficient for practical AIOps, because operators also need interpretable evidence and root cause ranking over candidate nodes or links.

Recent studies on multivariate time-series anomaly detection have demonstrated the effectiveness of deep representation learning for modeling complex dependencies. Representative methods include stochastic recurrent models such as OmniAnomaly[1], signature-based multiscale correlation modeling such as MSCRED[2], reconstruction-based methods such as USAD[3], one-class architectures such as THOC[4], graph-aware detectors such as MTAD-GAT[5] and GDN[6], hierarchical inter-metric modeling such as InterFusion[7], and Transformer-based approaches such as Anomaly Transformer[8], TranAD[9], and DCdetector[10]. These methods significantly improve anomaly detection accuracy by learning temporal patterns and inter-metric correlations. Nevertheless, most of them primarily focus on identifying abnormal timestamps or variables, while providing limited support for topology-aware root cause node localization in operational networks.

In parallel, a large body of work has investigated root cause analysis in distributed and service-oriented systems. CauseInfer[11], MonitorRank[12], CloudRanger[13], MicroCause[14], TraceRCA[15], and Nezha[16] have shown that causal structures, dependency graphs, trace information, and observability signals can substantially improve root cause localization. However, many of these methods assume that abnormal components have already been identified, or rely on service traces and application-level observability that may not always be available in network operation environments. For network-oriented AIOps, where topology information is usually easier to obtain than fine-grained traces, it remains necessary to design a unified framework that directly connects multivariate KPI anomaly detection with topology-guided and interpretable root cause node localization.

Another important gap lies in explainability. In practical operation scenarios, operators are reluctant to trust a model that only outputs an anomaly score or a ranked node list without supporting evidence. Explainable learning techniques such as Integrated Gradients[17], SHAP[18], and TimeSHAP[19] provide principled ways to attribute model decisions to features and temporal segments, making them attractive for trustworthy AIOps. At the same time, graph representation learning methods such as GCN[20] and GAT[21], as well as spatio-temporal graph models such as DCRNN[22] and Graph WaveNet[23], suggest that structural priors can be effectively fused into deep temporal models. These advances motivate a more unified solution for network operations, where anomaly detection, topology-aware propagation modeling, and explainable root cause node localization are jointly considered.

To address the above issues, this paper develops a topology-aware and self-supervised framework for multivariate KPI anomaly detection and root cause node localization in network operations. The temporal Transformer, graph encoder, and reconstruction and forecasting objectives used in the framework are established components rather than standalone methodological contributions. The contribution of this work lies in coupling these components with an explicit operational-topology prior and a downstream diagnosis stage: the learned KPI residuals are aggregated into metric-, node-, and system-level anomaly evidence, while candidate root cause nodes are ranked according to normalized local anomaly intensity and directed topology-constrained propagation consistency. Residual-based KPI contribution scores are reported separately as post hoc diagnostic evidence and do not enter the ranking objective.

The resulting formulation differs from three closely related lines of work. First, self-supervised and contrastive multivariate time-series detectors primarily identify anomalous timestamps or variables, but do not rank root-cause nodes under an observed network topology. Second, graph-based time-series detectors generally learn dependencies among variables for detection, whereas our framework uses the supplied physical and logical topology both during representation learning and during post-alarm propagation analysis. Third, network and service RCA methods often start from alarms, traces, call graphs, or already-detected anomalous components; our framework instead connects KPI-based anomaly detection directly to topology-constrained candidate ranking. Thus, the claimed contribution is the integrated diagnostic formulation and its topology-guided coupling, not the novelty of the individual Transformer, graph, reconstruction, or forecasting modules.

The main contributions of this paper are summarized as follows:

- We formulate a unified diagnostic pipeline that connects self-supervised multivariate KPI anomaly detection directly to topology-prior root cause node localization, rather than treating detection and diagnosis as independent tasks.
- We couple masked reconstruction and one-step prediction with topology-regularized temporal–structural representations, allowing operational topology to constrain both representation learning and downstream diagnosis.
- We define a root cause ranking strategy that combines normalized node-level anomaly evidence with directed, multi-hop propagation consistency under an explicit operational-topology prior; residual KPI shares are reported separately as non-causal diagnostic evidence.
- We establish a reproducible Mininet-based evaluation protocol covering three topology scales and six representative fault types. Across five seeds, the framework obtains an F1-score of 0.9235 ± 0.0037 and an A@1 of 0.7613 ± 0.0163. Under the controlled KPI-plus-topology observability budget, the latter is 5.47 percentage points higher than the restricted-input Nezha adaptation and does not constitute a comparison with native multimodal Nezha. The ablation study confirms the contribution of directed propagation to root-cause node localization.

The remainder of this paper is organized as follows. Section "Related work" reviews related work on multivariate time-series anomaly detection, topology-aware modeling, root cause localization, and explainable AIOps. Section "Problem formulation" formulates the problem and introduces the data representation. Section "Proposed

method" presents the proposed framework in detail. Section "Experiments" reports experimental results and ablation studies. Finally, Section "Conclusion and discussion" concludes the paper and discusses future research directions.

Related work

This section reviews the literature most relevant to our study from four perspectives: multivariate time-series anomaly detection, topology-aware and graph-based modeling, root cause localization in operational systems, and explainability for trustworthy AIOps. Consistent with the motivation in section "Introduction", our goal is not merely to improve anomaly detection accuracy, but to bridge anomaly discovery, topology-aware diagnosis, and interpretable decision support within a unified framework for network operations.

Multivariate time-series anomaly detection

Anomaly detection for multivariate time-series has attracted extensive attention due to its importance in cyber-physical systems, cloud platforms, industrial monitoring, and IT operations. Early approaches mainly relied on statistical modeling, matrix decomposition, or rule-based thresholding, but these methods often struggle with nonlinear dependencies, concept drift, and high-dimensional correlations. More recent deep learning methods have significantly improved detection performance by learning normal patterns from historical observations.

A representative line of work models multivariate anomalies through reconstruction or probabilistic generation. OmniAnomaly[1] employs stochastic recurrent neural networks and variational inference to capture complex temporal distributions and reconstruct normal observations. MSCRED[2] transforms sensor correlations into signature matrices and applies convolutional recurrent learning to detect abnormal behaviors. USAD[3] introduces an unsupervised adversarial framework based on dual autoencoders for robust anomaly scoring. These methods demonstrate that reconstruction-based learning is effective for unsupervised or weakly supervised anomaly detection, especially when labeled anomalies are scarce.

Another important direction focuses on learning hierarchical or one-class representations of normality. THOC[4] proposes a temporal hierarchical one-class network to capture multi-scale temporal patterns and improve anomaly sensitivity. InterFusion[7] further models inter-metric and temporal dependencies in a hierarchical latent structure, which is particularly suitable for operational metrics with strong coupling effects. These studies support the idea that robust anomaly detection requires not only temporal modeling but also explicit treatment of cross-metric interactions.

Recently, graph-based and attention-based models have become increasingly influential in multivariate anomaly detection. MTAD-GAT[5] combines forecasting and reconstruction with graph attention to capture relationships among variables. GDN[6] explicitly learns dependency structures among metrics and shows that graph-aware modeling can substantially improve anomaly detection performance. Transformer-based detectors have further advanced this line of work. Anomaly Transformer[8] exploits association discrepancy to separate abnormal points from normal temporal patterns, while TranAD[9] introduces a Transformer architecture tailored to anomaly detection in operational data streams. More recent methods such as DCdetector[10] continue this trend by refining contrastive and contextual representations for multivariate anomaly identification.

Although these methods have substantially improved anomaly detection accuracy, most existing studies emphasize *whether* a timestamp or variable is abnormal, rather than *where* the root cause lies in a networked system. In other words, the majority of current multivariate anomaly detection methods are detection-oriented rather than diagnosis-oriented. This limitation motivates the first part of our framework: learning anomaly-sensitive representations from unlabeled multivariate KPI data, while preserving information that can later support topology-aware root cause node localization.

Topology-aware and graph-based modeling for operational data

In many operational environments, especially network systems, variables are not independent observations but are embedded in explicit or implicit structural relationships. These relationships may arise from physical connectivity, logical dependencies, traffic paths, routing hierarchies, or shared resources. Consequently, topology-aware modeling has become an important research direction for improving both anomaly detection and diagnosis.

Graph neural networks provide a natural mechanism for representing such dependencies. Graph Convolutional Networks (GCN)[20] and Graph Attention Networks (GAT)[21] offer generic tools for propagating and aggregating information over graph structures. In the context of temporal and spatio-temporal data, DCRNN[22] and Graph WaveNet[23] demonstrate that combining graph structure with temporal modeling can effectively capture dynamic dependencies and propagation patterns. These models suggest that structural priors are not only useful for prediction tasks, but also potentially beneficial for identifying abnormal propagation in interconnected systems.

Within anomaly detection, MTAD-GAT[5] and GDN[6] provide strong evidence that graph-based dependency modeling improves multivariate anomaly detection by enhancing inter-variable representation learning. At the same time, Transformer-style sequence models such as "Attention Is All You Need"[24], Autoformer[25], and recent time-series foundation-style models such as PatchTST[26] show that long-range temporal dependencies can be captured more effectively than in conventional recurrent architectures. For network operations, however, temporal attention alone is insufficient, because abnormal propagation is often constrained by topology. Therefore, a practically useful AIOps framework should incorporate both temporal dependency learning and structural priors.

Nevertheless, existing topology-aware anomaly detection methods still exhibit two major limitations. First, many of them infer latent dependencies purely from data, without explicitly integrating known network topology or operational prior knowledge. Second, even when graph structure is used, the output is typically an anomaly

score rather than a root cause node ranking with interpretable propagation evidence. These gaps motivate the second part of our framework: using topology priors not only to regularize anomaly representation learning, but also to guide the downstream root cause node localization process.

Root cause node localization in operational systems

In the component-level setting considered here, root cause node localization aims to identify the most likely source node after abnormal system behavior has been observed. Compared with anomaly detection, RCA is more directly aligned with operational needs, because it supports troubleshooting, mitigation, and recovery actions. A large body of prior work has explored RCA in service-oriented, cloud-native, and distributed systems by leveraging performance metrics, traces, dependency graphs, or causal structures.

CauseInfer[11] investigates causal inference for performance diagnosis, while MonitorRank[12] formulates fault localization as a ranking problem over monitored components. CloudRanger[13] combines dependency information, anomaly detection, and random-walk-style analysis over an impact graph to rank root causes in cloud systems. MicroCause[14] improves fine-grained diagnosis for microservice systems through metric-level causal reasoning. TraceRCA[15] leverages traces to improve root cause localization and also provides informative evaluation measures such as top-k accuracy and ranking-based diagnosis quality. More recently, Nezha[16] further explores fine-grained RCA using multimodal observability data.

Network-oriented studies provide a complementary perspective. Kim *et al.*[27] combine unsupervised KPI anomaly detection with finite-state-machine analysis of message patterns and error codes in a mobile network. Jia *et al.*[28] use Transformer-based alarm context representations for root cause identification in optical networks. These studies demonstrate the value of network-specific monitoring and alarm context, but they do not use an explicit physical or logical topology to connect self-supervised multivariate KPI detection with multi-hop root cause ranking.

These studies clearly show that structural information and propagation analysis are critical for accurate diagnosis. However, many RCA methods assume the availability of service traces, application call graphs, or already-detected anomalous components. Such assumptions may not hold in network operation scenarios, where topology information and KPI monitoring are usually easier to obtain than fine-grained end-to-end traces. Moreover, a substantial portion of RCA work focuses on service systems rather than network-centric environments, leaving a gap between existing diagnosis techniques and the operational characteristics of communication networks.

For our target problem, root cause node localization should be tightly coupled with the anomaly detection stage. Specifically, detection should provide fine-grained abnormal evidence at the metric and node levels, while topology-aware RCA should exploit connectivity constraints and abnormal propagation consistency to rank candidate root cause nodes. This perspective distinguishes our framework from prior RCA studies that treat anomaly discovery and diagnosis as loosely connected stages.

Explainability for trustworthy AIOps

Explainability is essential for practical AIOps deployment. In real-world network operation settings, operators must decide whether to trust a model-generated alarm, whether to inspect a suggested root cause, and whether to initiate a mitigation procedure. Therefore, a diagnosis system that outputs only a scalar score or a ranked list is often insufficient. Instead, it should provide interpretable evidence, such as which KPI dimensions contributed most to the alarm, which temporal segments were most influential, and how the abnormal behavior propagated across the topology.

Feature attribution methods offer a principled way to support such explanations. Integrated Gradients[17] provides an axiomatic attribution framework for differentiable models. SHAP[18] unifies several feature importance approaches through game-theoretic explanations and has been widely used in interpretable machine learning. TimeSHAP[19] extends Shapley-style reasoning to sequential prediction settings and is particularly relevant for time-series applications. These methods indicate that explanation can be performed not only at the feature level, but also at the temporal segment level, which is highly desirable for KPI-based AIOps.

Despite this progress, explainability has only been partially integrated into anomaly detection and RCA research. In many studies, explainability is treated as an afterthought for post hoc interpretation. For network operations, an AIOps system can provide two complementary forms of inspectable evidence: *residual-based KPI contributions*, which summarize which observed KPI errors support a node anomaly, and *topology-level propagation evidence*, which shows why a node is ranked as a plausible root cause. Unlike Integrated Gradients or SHAP, the former is not a model-decision attribution. This distinction motivates the deliberately limited explanation scope of our framework.

Summary and positioning of this work

In summary, the existing literature provides the main building blocks used in this study. The distinction is how these blocks are coupled and what output they produce. Table 1 compares the proposed framework with representative self-supervised or contrastive MTS anomaly detectors, graph-based time-series detectors, and service- or network-oriented RCA methods. The comparison does not claim that the individual temporal encoder, graph encoder, or self-supervised objectives are new. Instead, it identifies the specific gap addressed here: jointly producing KPI anomaly detection and root cause ranking from KPI streams while using an observed operational topology in both representation learning and propagation-based diagnosis.

Accordingly, this work is positioned as an integration contribution at the intersection of self-supervised MTS anomaly detection, topology-aware representation learning, and root cause node localization. Its distinguishing feature is the use of the same observed topology to constrain learned node representations and to evaluate

Method group	Representative methods	Learning signal	Explicit topology	Primary output	Diagnostic evidence
Self-supervised/ contrastive MTS-AD	Anomaly Transformer, TranAD, DCdetector[8–10]	Association, reconstruction, or contrastive objectives	No	Anomalous timestamps or variables	Model- or score-based variable evidence
Graph-based TSAD	MTAD-GAT and GDN[5,6]	Forecasting and/or reconstruction	Learned variable graph	Anomaly scores	Learned dependencies or attention weights
Service-system RCA	CloudRanger, TraceRCA, and Nezha[13,15,16]	Alarm, metric, trace, or multimodal evidence	Service dependency or call graph	Root-cause ranking	Causal, random-walk, trace, or multimodal evidence
Network-oriented diagnosis	Mobile-network RCA and optical-alarm RCA[27,28]	KPI/message patterns or alarm context	No explicit topology in the compared formulation	Anomaly/RCA output or alarm root cause	Message patterns, error codes, or alarm context
Proposed framework	This work	Masked reconstruction, one-step prediction, and topology regularization	Physical and logical network topology	KPI anomaly detection and root cause ranking	Metric-level evidence and multi-hop propagation subgraph

Table 1. Positioning of the proposed framework relative to representative anomaly-detection and root-cause-localization approaches. "Explicit topology" denotes a supplied physical, logical, service, or call-dependency graph rather than a graph learned only from variable correlations.

whether post-alarm anomaly patterns are consistent with multi-hop propagation from a candidate root cause node.

Problem formulation

This section formulates the problem studied in this paper. We first define the network topology and the multivariate KPI representation. We then describe the anomaly detection and root cause node localization tasks in a unified way. Finally, we present the learning objective and summarize the experimental scope and main notations.

Network topology and KPI representation

We model a network operation environment as a graph

$$G_{\text{enc}} = (V, E_{\text{enc}}, A_{\text{enc}}), \qquad G_{\text{rca}}^{(t)} = (V, E_{\text{rca}}^{(t)}, W^{(t)}), \tag{1}$$

where $V = \{v_1, v_2, \ldots, v_N\}$ is the set of monitored nodes. $A_{\text{enc}} \in \mathbb{R}^{N\times N}$ is the adjacency matrix used by the representation encoder and topology regularizer. In contrast, $W^{(t)} \in \mathbb{R}_{\geq 0}^{N\times N}$ is the directed nonnegative propagation matrix used only by the RCA module: $W_{ij}^{(t)} > 0$ means that an observed forwarding or routing relation permits a fault effect to propagate from a candidate source v_i to a potentially affected node v_j. Separating these matrices avoids interpreting generic neighborhood similarity in the encoder as a directed propagation relation. A physical undirected link is represented by two reciprocal directed edges when no forwarding direction is available; an unavailable edge weight defaults to one. Self-edges are excluded from $W^{(t)}$.

The time index permits $W^{(t)}$ to be updated when a new routing or forwarding snapshot is available. The present Mininet evaluation uses the forwarding relations recorded for each fault event and otherwise treats the topology as piecewise static. Link capacity or other reliable numerical link attributes can be stored in $W^{(t)}$, but the current evaluation does not separately encode edge type, propagation delay, or continuously changing routes. We therefore treat support for these attributes as an interface of the formulation rather than an empirically validated capability, and return to this limitation in Section "Conclusion and discussion".

For each node v_i, the monitoring system collects a d-dimensional KPI vector at time step t:

$$\mathbf{x}_i^{(t)} = [x_{i,1}^{(t)}, x_{i,2}^{(t)}, \ldots, x_{i,d}^{(t)}]^\top \in \mathbb{R}^d, \tag{2}$$

where the KPI dimensions may include CPU utilization, memory usage, inbound traffic, outbound traffic, RTT, packet loss, queue length, and interface error count.

Over a temporal window of length T, the KPI observations of the whole network are written as

$$\mathcal{X}_t = \{\mathbf{X}^{(t-T+1)}, \mathbf{X}^{(t-T+2)}, \ldots, \mathbf{X}^{(t)}\} \in \mathbb{R}^{T\times N\times d}, \tag{3}$$

where

$$\mathbf{X}^{(t)} = [\mathbf{x}_1^{(t)}, \mathbf{x}_2^{(t)}, \ldots, \mathbf{x}_N^{(t)}]^\top \in \mathbb{R}^{N\times d}. \tag{4}$$

We use a sliding-window setting throughout the paper. For a window ending at time t, the model input is the KPI tensor $\mathcal{X}_t$ together with A_{enc}; the post-alarm RCA stage additionally uses $W^{(t)}$. This representation is consistent with multivariate operational metric modeling in anomaly detection methods such as OmniAnomaly, MTAD-GAT, GDN, InterFusion, and Transformer-based detectors[1,5–10]. The main difference in our setting is that the operational topology prior is explicitly included because it is naturally available in network operations.

Problem definition

Given the KPI window $\mathcal{X}_t$, encoder topology A_{enc}, and directed RCA propagation prior $W^{(t)}$, the goal of this paper is to solve anomaly detection and root cause node localization in one unified framework.

The first task is **multivariate KPI anomaly detection**. The framework should identify whether the current window contains abnormal behavior and should output anomaly scores at different levels. These include KPI-level scores, node-level scores, and a system-level alarm score. This design allows the model to support both global alarm triggering and local fault inspection.

The second task is **root cause node localization**. Once an anomaly is detected, the framework ranks the most likely root cause nodes. The ranking should not depend only on local abnormality. It should also consider whether a candidate node can explain the observed abnormal spread under the topology prior.

The third task is **explainable diagnosis**. The framework should provide evidence for the final RCA result. This evidence should include the most important KPI dimensions for each candidate node and a topology-level explanation path that shows how the fault may propagate across the network.

Formally, we define the overall mapping of the framework as

$$f_\theta : (\mathcal{X}_t, A_{\text{enc}}, W^{(t)}) \mapsto \left(S^{(t)}, \mathbf{s}^{(t)}, \mathbf{R}^{(t)}, \mathbf{C}^{(t)}\right), \tag{5}$$

where $f_\theta(\cdot)$ is the proposed model with parameters θ, $S^{(t)}$ is the system-level anomaly score, $\mathbf{s}^{(t)} \in \mathbb{R}^N$ is the node-level anomaly score vector, $\mathbf{R}^{(t)}$ is the ranked root cause list, and $\mathbf{C}^{(t)}$ denotes the explanation output.

Accordingly, $\mathbf{R}^{(t)}$ identifies and ranks candidate source nodes; it does not assign a categorical fault-type label such as CPU overload, memory exhaustion, or link degradation. The KPI contribution vector in $\mathbf{C}^{(t)}$ indicates which observed metrics have the largest residual shares and may support operator interpretation, but it is not a trained fault-type classifier.

In practical network operations, the model should satisfy four requirements. It should work with limited fault labels, capture temporal and cross-metric dependencies, use topology information effectively, and generate explanations that are useful for operators. These requirements are consistent with the limitations of existing anomaly detection and RCA studies discussed in section "Related work"[1,6,13,15,17–19].

Learning objective

We focus on a self-supervised setting because high-quality fault labels are scarce in real network operation environments. The model is trained mainly on normal or weakly contaminated KPI windows and learns normal behavior patterns from the data itself.

To support self-supervised learning, we apply a point-wise masking operator to the normalized input window. During each training epoch, independent Bernoulli variables $M_{\tau ik} \sim \text{Bernoulli}(1-q)$ are sampled over time–node–KPI positions, with mask ratio $q = 0.15$, and masked normalized values are replaced by zero:

$$\tilde{x}_{i,k}^{(\tau)} = M_{\tau ik}\bar{x}_{i,k}^{(\tau)}. \tag{6}$$

The reconstruction loss is evaluated only on the sampled masked set $\Omega_{\text{mask}} = \{(\tau, i, k) : M_{\tau ik} = 0\}$, whereas the one-step prediction loss uses all KPI dimensions. Random masks are used only during training; validation and test anomaly scores are computed from unmasked windows. This learning strategy follows the main idea of reconstruction- and prediction-based anomaly detection[1,5,6,8,9].

For node v_i and KPI dimension k, let $\hat{x}_{i,k}^{(t)}$ be the reconstructed value and let $\tilde{x}_{i,k}^{(t+1)}$ be the one-step prediction. We define the KPI-level anomaly score as

$$s_{i,k}^{(t)} = \alpha\left|x_{i,k}^{(t)} - \hat{x}_{i,k}^{(t)}\right| + (1-\alpha)\left|x_{i,k}^{(t+1)} - \tilde{x}_{i,k}^{(t+1)}\right|, \tag{7}$$

where $\alpha \in [0,1]$ balances reconstruction error and prediction error.

The node-level anomaly score is obtained by aggregation:

$$s_i^{(t)} = \sum_{k=1}^{d} w_k s_{i,k}^{(t)}, \tag{8}$$

where w_k is the weight of KPI k. The system-level anomaly score is then written as

$$S^{(t)} = \phi\left(s_1^{(t)}, s_2^{(t)}, \ldots, s_N^{(t)}\right), \tag{9}$$

where $\phi(\cdot)$ is an aggregation operator.

Once an alarm is triggered, the framework performs root cause ranking. For a candidate node v_i, the RCA score is defined as

$$r_i^{(t)} = \gamma\bar{s}_i^{(t)} + \eta\bar{p}_i^{(t)}, \qquad \gamma, \eta \geq 0, \quad \gamma + \eta = 1, \tag{10}$$

where $\bar{s}_i^{(t)}$ and $\bar{p}_i^{(t)}$ are candidate-set-normalized local anomaly and propagation consistency scores. Residual-based KPI contributions are reported only as diagnostic evidence and are deliberately excluded from Eq. (10).

Equation (10) has only one free mixing coefficient because $\eta = 1 - \gamma$. Residual-based KPI contributions are computed only after ranking and are not inputs to the RCA score. We treat γ as a validation-selected operating coefficient rather than a trainable model parameter. Jointly learning it would require root-cause labels for the post-alarm ranking stage and would therefore add a supervised-label assumption to the otherwise self-supervised detector. The selected coefficient is fixed before test evaluation and shared across topology scales and fault types.

The ranked root cause list is

$$\mathbf{R}^{(t)} = \mathrm{rank}\left(\{r_i^{(t)}\}\right). \tag{11}$$

The final learning objective combines reconstruction, prediction, and topology consistency:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\mathrm{rec}} + \lambda_2 \mathcal{L}_{\mathrm{pred}} + \lambda_3 \mathcal{L}_{\mathrm{topo}}, \tag{12}$$

where $\mathcal{L}_{\mathrm{rec}}$ is the reconstruction loss, $\mathcal{L}_{\mathrm{pred}}$ is the prediction loss, and $\mathcal{L}_{\mathrm{topo}}$ is the topology regularization term. The explanation outputs are deterministic post hoc quantities and do not define an optimization loss. The exact form of these terms is given in section "Proposed method".

Figure 1 illustrates the problem setting considered in this paper.

Experimental scope and main notations

To keep the formulation consistent with the later experiments, Table 2 summarizes the experimental data scope and testbed configuration.

Table 3 lists the main symbols used in the remainder of the paper.

The formulation in this section defines the data representation, task objectives, and learning target of the proposed framework. Based on this formulation, the next section presents the topology-aware self-supervised detector, directed RCA rule, and post hoc evidence outputs in detail.

Proposed method

This section presents the proposed framework for multivariate KPI anomaly detection and root cause node localization in network operations. The framework takes multivariate KPI streams and topology priors as input. It learns normal operational patterns in a self-supervised way, detects anomalies at multiple levels, ranks root-cause node candidates using anomaly and propagation scores, and reports separate evidence for inspection. The whole design follows the problem setting in section "Problem formulation" and supports label-efficient anomaly detection, topology-aware diagnosis, and operational transparency.

Framework overview

The proposed framework contains five connected modules: input encoding, topology-aware self-supervised representation learning, multi-granularity anomaly scoring, topology-prior root cause node localization, and post hoc evidence generation. The model first learns the normal behavior of KPI streams from mostly unlabeled data. It then measures abnormality at the KPI, node, and system levels. Once an alarm is triggered, it ranks node

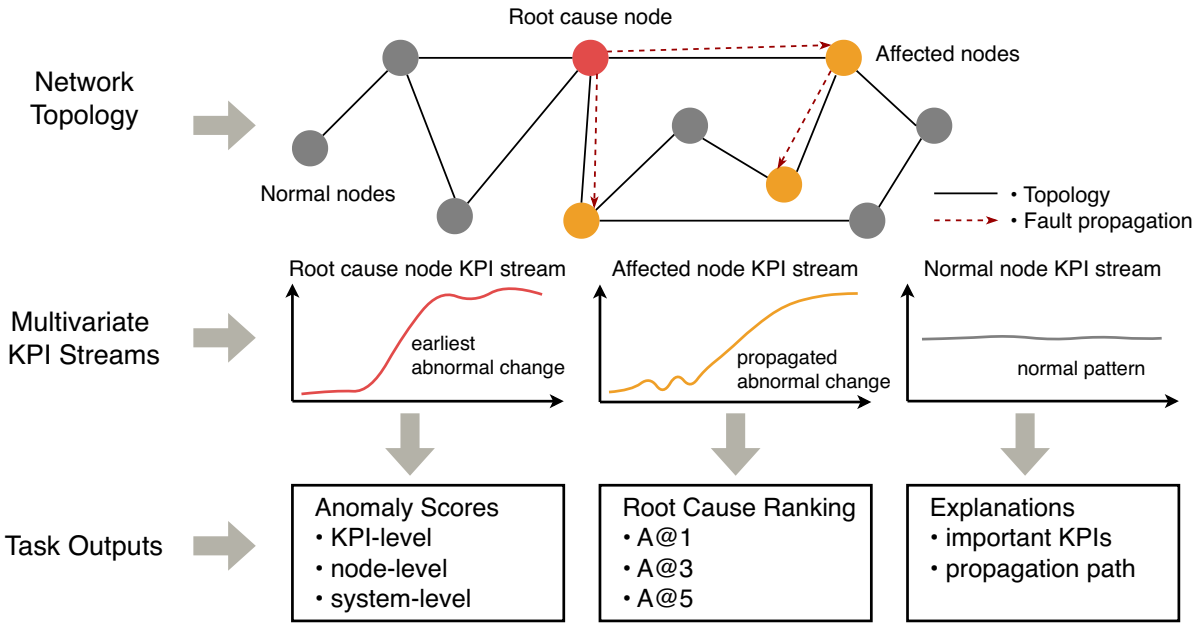


Fig. 1. Illustration of the problem setting. A root fault originates at one node, affects multivariate KPI streams, propagates through the network topology, and leads to anomaly detection, root cause ranking, and explanation outputs.

Item	Configuration/value
Topology scenarios	3 network topologies: small, medium, and large
Number of monitored nodes N	12, 24, and 48 nodes
Node types	Router/switch/server/virtualized network component
Edge types	Physical links and logical forwarding/dependency links
Number of KPIs per node d	8 core KPIs
Core KPI set	CPU, memory, inbound traffic, outbound traffic, RTT, packet loss, queue length, and interface errors
Sampling interval	30 s
Window length T	20 time steps (10 min)
Window stride	1 time step
Training data assumption	Predominantly normal data with a controlled 3% abnormal-window contamination ratio
Validation set size	About 15% of all windows
Test set size	About 25% of all windows
Fault categories	CPU overload, bandwidth bottleneck, packet loss, link delay, interface failure, and routing instability
Fault injection granularity	Single-root and multi-hop propagation cases
Anomaly ratio in controlled test set	8%–15% abnormal windows (event-enriched evaluation)
RCA target	A@1/A@3/A@5 root-cause node localization accuracy
Explanation output	Top contributing KPIs and topology-consistent propagation path

Table 2. Experimental data scope and testbed configuration.

Symbol	Description	Symbol	Description
G_{enc}	Encoder topology graph	$\mathbf{x}_i^{(t)}$	KPI vector of node v_i at time t
$G_{\text{rca}}^{(t)}$	Directed RCA propagation graph	$\mathbf{X}^{(t)}$	Network-wide KPI matrix at time t
V	Set of monitored nodes	$\mathcal{X}_t$	KPI tensor over the current input window
A_{enc}	Encoder adjacency matrix	$s_{i,k}^{(t)}$	KPI-level anomaly score
$W^{(t)}$	Directed nonnegative RCA edge-weight matrix	$s_i^{(t)}$	Node-level anomaly score
$P^{(t)}$	Row-normalized RCA propagation matrix	$S^{(t)}$	System-level anomaly score
$\pi_{ij}^{(t)}$	Candidate-normalized multi-hop influence from i to j	$r_i^{(t)}$	Root cause node localization score of candidate node v_i
N	Number of monitored nodes	$c_{i,k}^{(t)}$	Residual-based KPI contribution
d	Number of KPI dimensions per node	$\mathbf{R}^{(t)}$	Ranked root cause list
T	Input window length	$\mathbf{C}^{(t)}$	Post hoc diagnostic evidence output

Table 3. Main notations used in this paper.

candidates using normalized local anomaly and directed propagation consistency, and subsequently reports KPI-residual shares and a propagation subgraph for inspection.

This design differs from prior studies that focus only on anomaly detection[1–10] or only on root cause localization[11–16]. Our framework connects these two tasks in a single model and uses explanation as part of the final output.

Fig. 2 shows the overall workflow of the proposed method.

Topology-aware self-supervised representation learning

The first part of the framework learns node representations from multivariate KPI windows and topology priors. This part is the backbone of the whole method. It should capture temporal dynamics, cross-metric relationships, and structural dependencies among nodes. It should also work without requiring a large amount of labeled fault data.

Given an input KPI window $\mathcal{X}_t \in \mathbb{R}^{T\times N\times d}$ and encoder adjacency matrix $A_{\text{enc}} \in \mathbb{R}^{N\times N}$, we first normalize each KPI dimension:

$$\tilde{x}_{i,k}^{(t)} = \frac{x_{i,k}^{(t)} - \mu_k}{\sigma_k + \epsilon}, \tag{13}$$

where μ_k and σ_k are estimated from the training set and ϵ is a small constant. This step removes scale differences across KPI dimensions and improves training stability.

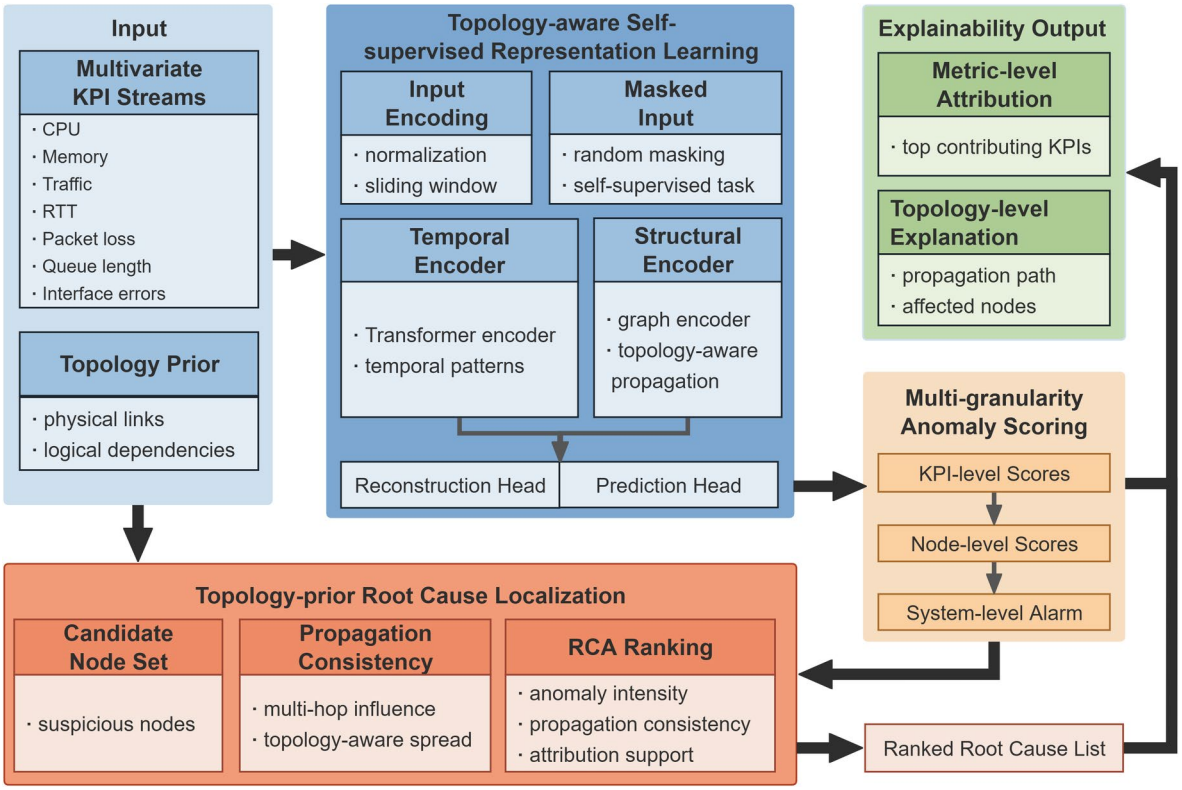


Fig. 2. Overall framework of the proposed topology-aware and self-supervised method. Root-cause ranking uses only normalized anomaly and directed propagation scores; residual-based KPI contributions and the propagation subgraph are detached post hoc evidence.

For node v_i, its KPI sequence in the current window is written as

$$\mathbf{Z}_i = [\tilde{\mathbf{x}}_i^{(t-T+1)}, \tilde{\mathbf{x}}_i^{(t-T+2)}, \ldots, \tilde{\mathbf{x}}_i^{(t)}] \in \mathbb{R}^{T\times d}. \tag{14}$$

For representation learning, we add self-loops to A_{enc} and use symmetric normalization $\tilde{A}_{\text{enc}} = D^{-1/2}(A_{\text{enc}} + I)D^{-1/2}$, where $D_{ii} = \sum_j (A_{\text{enc}} + I)_{ij}$. This encoder matrix is distinct from the directed RCA propagation matrix defined below. To inject structural information from the topology, we aggregate neighborhood context and obtain the initial topology-aware representation:

$$\mathbf{H}_i^{(0)} = \mathbf{Z}_i + \beta \sum_{j=1}^{N} (\tilde{A}_{\text{enc}})_{ij} \mathbf{Z}_j, \tag{15}$$

where β controls the strength of the topology prior. This design preserves the local KPI sequence of each node and adds information from its neighbors. It follows the intuition of graph-based operational modeling in GCN, GAT, MTAD-GAT, and GDN[5,6,20,21].

We adopt the point-wise masking scheme defined in Eq. (6). A new Bernoulli mask is sampled for every training window and epoch, zero is the training-set mean in normalized coordinates, and reconstruction error is accumulated only over Ω_{mask}. The model is trained to reconstruct the masked values and predict future observations. This allows it to learn normal operational patterns from the data itself, which is consistent with prior self-supervised anomaly detection studies[1,5,6,8,9].

To model temporal dependencies, we use a Transformer-style temporal encoder. Given the topology-aware input representation, the query, key, and value matrices are defined as

$$\mathbf{Q} = \mathbf{H}^{(0)} W_Q, \quad \mathbf{K} = \mathbf{H}^{(0)} W_K, \quad \mathbf{V} = \mathbf{H}^{(0)} W_V, \tag{16}$$

and the attention output is

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}}\right)\mathbf{V}, \tag{17}$$

where W_Q, W_K, and W_V are learnable projection matrices and d_h is the hidden dimension. This encoder captures both short-term and long-range patterns in KPI sequences. It is motivated by Transformer-based time-series models[24–26].

Temporal modeling alone is not enough in a network setting. Abnormal behavior often spreads through connected nodes. We therefore add a structural encoder after temporal encoding. A simple graph convolution form is

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{A}_{\text{enc}}\mathbf{H}^{(l)}W^{(l)}\right), \tag{18}$$

where $\mathbf{H}^{(l)}$ is the hidden representation at layer l, $W^{(l)}$ is the learnable weight matrix, and $\sigma(\cdot)$ is an activation function. This step propagates information over the topology graph and keeps node representations structurally aware.

The final latent representation is sent to two task heads. The first head reconstructs the masked values:

$$\hat{\mathcal{X}}_t = g_{\text{rec}}(\mathbf{H}^{(L)}), \tag{19}$$

and the second head predicts the next-step KPI values:

$$\tilde{\mathbf{X}}^{(t+1)} = g_{\text{pred}}(\mathbf{H}^{(L)}). \tag{20}$$

The corresponding reconstruction and prediction losses are

$$\mathcal{L}_{\text{rec}} = \frac{1}{|\Omega_{\text{mask}}|}\sum_{(i,k,\tau)\in\Omega_{\text{mask}}}\left|x_{i,k}^{(\tau)} - \hat{x}_{i,k}^{(\tau)}\right|, \tag{21}$$

$$\mathcal{L}_{\text{pred}} = \frac{1}{Nd}\sum_{i=1}^{N}\sum_{k=1}^{d}\left|x_{i,k}^{(t+1)} - \tilde{x}_{i,k}^{(t+1)}\right|. \tag{22}$$

To keep the learned embeddings consistent with the topology under normal conditions, we add a topology regularization term:

$$\mathcal{L}_{\text{topo}} = \frac{1}{Z_A}\sum_{i=1}^{N}\sum_{j=1}^{N}(\tilde{A}_{\text{enc}})_{ij}\left\|\mathbf{h}_i - \mathbf{h}_j\right\|_2^2, \qquad Z_A = \sum_{i=1}^{N}\sum_{j=1}^{N}(\tilde{A}_{\text{enc}})_{ij}, \tag{23}$$

where $\mathbf{h}_i$ and $\mathbf{h}_j$ are the final latent representations of nodes i and j, and Z_A normalizes the loss by the total adjacency mass. This normalization prevents the magnitude of the topology term from increasing merely because the topology contains more nodes or edges and makes a shared regularization weight meaningful across the 12-, 24-, and 48-node settings. This term is a soft neighborhood prior rather than a constraint that forces adjacent representations to be identical. The node's own KPI sequence is retained through the residual local input in Eq. (15), and the reconstruction and prediction objectives continue to penalize loss of node-specific behavior. Nevertheless, the present regularizer treats every nonzero entry of $\tilde{A}_{\text{enc}}$ as the same type of compatibility relation. In a strongly heterogeneous topology, adjacent routers, switches, servers, or virtualized components may have legitimately different normal representations; penalizing those differences can cause cross-role oversmoothing. The current formulation therefore assumes that physical adjacency provides a useful but imperfect similarity prior and does not establish robustness to arbitrary node-role heterogeneity. A role-aware extension could replace $(\tilde{A}_{\text{enc}})_{ij}$ by a type-compatibility-gated weight or compare relation-specific projected embeddings before regularization. The overall self-supervised learning objective is

$$\mathcal{L}_{\text{SSL}} = \lambda_1\mathcal{L}_{\text{rec}} + \lambda_2\mathcal{L}_{\text{pred}} + \lambda_3\mathcal{L}_{\text{topo}}. \tag{24}$$

We set $\lambda_1 = \lambda_2 = 1$ so that the two element-wise mean absolute-error objectives define the reference scale. The topology term is adjacency-mass normalized as in Eq. (23), and λ_3 is selected using only the validation split. The candidate values are 0, 0.01, 0.05, 0.10, 0.20, and 0.50. The selected value $\lambda_3 = 0.10$ provides the validation-selected balance between retaining the KPI reconstruction and prediction signal and introducing a non-dominant topology prior. It is fixed before test evaluation and shared across topology scales and fault types.

This module produces topology-aware node embeddings that support both anomaly detection and root cause ranking in later stages.

Multi-granularity anomaly scoring

After learning the latent representation, the framework converts reconstruction and prediction errors into anomaly scores. We use three levels of scoring because network operators need to know not only whether the whole system is abnormal, but also which nodes and which KPI dimensions are responsible.

At the KPI level, the anomaly score for node v_i and KPI dimension k is defined as

$$s_{i,k}^{(t)} = \alpha\left|x_{i,k}^{(t)} - \hat{x}_{i,k}^{(t)}\right| + (1-\alpha)\left|x_{i,k}^{(t+1)} - \tilde{x}_{i,k}^{(t+1)}\right|, \tag{25}$$

where α balances reconstruction error and prediction error. This score captures both current mismatch and short-term future deviation.

At the node level, we aggregate the KPI-level scores:

$$s_i^{(t)} = \sum_{k=1}^{d} w_k s_{i,k}^{(t)}, \tag{26}$$

where the KPI weights are defined as

$$w_k = \frac{\exp(\rho_k)}{\sum_{m=1}^{d} \exp(\rho_m)}. \tag{27}$$

This weighted aggregation reduces the effect of noisy single metrics and produces a compact anomaly score for each node.

At the system level, we use top-m aggregation:

$$S^{(t)} = \frac{1}{m} \sum_{i \in \mathrm{Top}-m(\mathbf{s}^{(t)})} s_i^{(t)}, \tag{28}$$

where $\mathrm{Top}-m(\mathbf{s}^{(t)})$ is the set of the m nodes with the highest node scores. This design is suitable for network operations because many faults begin as local events. A simple global average may weaken the signal of such local failures. The alarm indicator is defined as

$$\mathbb{I}_{\mathrm{alarm}}^{(t)} = \begin{cases} 1, & S^{(t)} > \tau, \\ 0, & \text{otherwise}, \end{cases} \tag{29}$$

where τ is the threshold estimated from the validation set.

This stage outputs KPI-level scores, node-level scores, and a system-level alarm signal. These outputs are then used by the RCA module.

Topology-prior root cause node localization

Once an alarm is triggered, the framework enters the diagnosis stage. The goal here is not simply to choose the most abnormal node. A true root cause should also explain why the other abnormal nodes appear under the topology prior.

We first construct the thresholded candidate set and cap its size at K:

$$\mathcal{C}^{(t)} = \mathrm{Top}-K\left(\left\{v_i \in V \mid s_i^{(t)} > \tau_n\right\}\right). \tag{30}$$

Here τ_n is selected on the validation split by maximizing node-level A@1 after the system alarm, and $K = 5$ is fixed before test evaluation. If a system alarm is raised but no node exceeds τ_n, the highest-scoring node is retained as a deterministic fallback.

The directed propagation matrix is row normalized as

$$P_{ij}^{(t)} = \begin{cases} \dfrac{W_{ij}^{(t)}}{\sum_{q=1}^{N} W_{iq}^{(t)}}, & \sum_q W_{iq}^{(t)} > 0, \\ 0, & \text{otherwise}. \end{cases} \tag{31}$$

Thus, an isolated node has an all-zero row and cannot receive propagation support from unreachable candidates. For a maximum propagation depth L_p, the unnormalized multi-hop influence is

$$u_{ij}^{(t)} = \sum_{\ell=1}^{L_p} \delta^{\ell-1} \left[(P^{(t)})^{\ell}\right]_{ij}, \qquad 0 < \delta \leq 1, \tag{32}$$

where δ is a single hop-decay parameter. The geometric sequence $1, \delta, \ldots, \delta^{L_p-1}$ constrains influence to decrease monotonically with hop distance and represents the L_p hop weights with one validation-selected parameter. We use one shared δ across all fault categories because the fault type is not known when RCA ranking is performed; selecting a type-specific decay from the true fault label would leak evaluation information. The shared value is selected on validation events containing all six fault categories and is fixed before test evaluation. We then normalize influence only over other abnormal candidates:

$$\pi_{ij}^{(t)} = \frac{\mathbb{I}[i \neq j] u_{ij}^{(t)}}{\sum_{q \in \mathcal{C}^{(t)} \setminus \{i\}} u_{iq}^{(t)} + \epsilon}. \tag{33}$$

When the denominator is zero, every $\pi_{ij}^{(t)}$ in that row is defined as zero. Otherwise, $\sum_{j\in\mathcal{C}^{(t)}} \pi_{ij}^{(t)} = 1$. The propagation consistency score is

$$p_i^{(t)} = \sum_{j\in\mathcal{C}^{(t)}} \pi_{ij}^{(t)} s_j^{(t)}. \tag{34}$$

Because local and propagation scores need not share a scale, both are normalized over the same candidate set:

$$\bar{s}_i^{(t)} = \frac{s_i^{(t)}}{\sum_{q\in\mathcal{C}^{(t)}} s_q^{(t)} + \epsilon}, \qquad \bar{p}_i^{(t)} = \frac{p_i^{(t)}}{\sum_{q\in\mathcal{C}^{(t)}} p_q^{(t)} + \epsilon}. \tag{35}$$

The final RCA score is

$$r_i^{(t)} = \gamma \bar{s}_i^{(t)} + \eta \bar{p}_i^{(t)}, \qquad \gamma + \eta = 1. \tag{36}$$

Thus, γ is the only free RCA mixing coefficient and $\eta = 1 - \gamma$. Residual-based KPI contributions are detached post hoc evidence and do not affect the candidate ranking. We do not optimize γ jointly with the detector because doing so would require root-cause labels during model training. Instead, γ is selected using node-level A@1 on the validation split, then fixed for every topology scale and fault type in the test set.

The ranked root cause list is

$$\mathbf{R}^{(t)} = \mathrm{rank}\left(\{r_i^{(t)} \mid v_i \in \mathcal{C}^{(t)}\}\right). \tag{37}$$

This RCA design combines local abnormality and directed propagation consistency while keeping the diagnostic explanation separate from the ranking rule. It is closely connected to the anomaly detection stage and differs from methods that treat diagnosis as a completely separate process. It also follows the main idea of ranking-based RCA methods such as CloudRanger and TraceRCA, while adapting them to KPI-based network operations[13,15].

Explainability and optimization

The final part of the framework provides post hoc evidence for the RCA result and defines the overall training objective. Diagnostic transparency is important in network operations because operators need evidence that supports the ranked diagnosis result.

At the metric level, we compute a residual-based KPI contribution score for each KPI dimension:

$$c_{i,k}^{(t)} = \frac{s_{i,k}^{(t)}}{\sum_{m=1}^{d} s_{i,m}^{(t)} + \epsilon}. \tag{38}$$

This produces the contribution vector

$$\mathbf{c}_i^{(t)} = [c_{i,1}^{(t)}, c_{i,2}^{(t)}, \ldots, c_{i,d}^{(t)}]. \tag{39}$$

The vector reports the relative share of reconstruction and prediction residuals across KPI dimensions. It is descriptive evidence for the observed anomaly, not a causal explanation or a model-gradient feature attribution such as Integrated Gradients, SHAP, or TimeSHAP[17–19]. It is not used to compute $r_i^{(t)}$.

At the topology level, we extract an explanation subgraph:

$$G_{\mathrm{exp}}^{(t)} = (V_{\mathrm{exp}}^{(t)}, E_{\mathrm{exp}}^{(t)}), \tag{40}$$

where $V_{\mathrm{exp}}^{(t)}$ contains the top-ranked candidate and the most affected abnormal nodes, and $E_{\mathrm{exp}}^{(t)}$ contains directed propagation edges selected according to $\pi_{ij}^{(t)}$. This subgraph gives a compact description of how the fault may spread across the available topology prior and makes the RCA result easier to inspect.

The training objective excludes an explanation-consistency term because a target computed from an RCA score that already contains the explanation quantity would create a circular objective. The objective is therefore

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\mathrm{rec}} + \lambda_2 \mathcal{L}_{\mathrm{pred}} + \lambda_3 \mathcal{L}_{\mathrm{topo}}, \tag{41}$$

and no explanation term or λ_4 remains. We use $\lambda_1 = \lambda_2 = 1$ and select $\lambda_3 = 0.10$ on the validation split from candidate values 0, 0.01, 0.05, 0.10, 0.20, and 0.50; the selected value is fixed before test evaluation. During training, the reconstruction, prediction, and topology losses are optimized jointly with Adam. Early stopping monitors validation loss with patience eight. The residual contribution vector and explanation subgraph are computed only after anomaly scores have been produced at inference. In implementation, the score tensors are detached before this deterministic post-processing; consequently, neither $\mathbf{c}_i^{(t)}$ nor $G_{\mathrm{exp}}^{(t)}$ creates a gradient path to the detector or RCA module.

Validation data are used to select τ by system-level window-based F1-score, τ_n by post-alarm node-level A@1, the topology regularization weight λ_3, the top-m aggregation size, δ, and the RCA weight γ (with $\eta = 1 - \gamma$). All

selected values are fixed before test evaluation. During inference, the framework first collects and normalizes KPI data. It then builds topology-aware input features and computes temporal–structural node representations. The model reconstructs the unmasked current window and predicts future values. These outputs are converted into KPI-level, node-level, and system-level anomaly scores. If the alarm threshold is exceeded, the RCA module builds a candidate node set, computes normalized RCA scores, and outputs the ranked root cause list together with KPI-contribution and topology-path evidence.

Table 4 lists the default implementation settings of the framework. These settings are used in the experiments and are consistent with the data scale defined in section "Problem formulation".

Experiments

This section evaluates the proposed framework from six aspects: anomaly detection, root cause node localization, cross-system external validation, ablation and stability, explainability, and computational efficiency. The experiments are designed to answer six questions. First, can the proposed framework detect anomalies from unlabeled multivariate KPI streams more accurately than representative baselines? Second, can the topology-prior diagnosis module improve root cause node localization quality? Third, does the observed performance transfer to a public semi-real benchmark under the stated adaptation protocol? Fourth, how much does each component contribute to the final performance? Fifth, can the framework provide useful metric-level and topology-level diagnostic evidence for operators? Sixth, what runtime and memory costs are observed under the evaluated hardware and topology scales?

Experimental setup

We evaluate the proposed framework on a network operation testbed built on Mininet[29]. The testbed includes three topology scales, namely small, medium, and large settings with 12, 24, and 48 monitored nodes. Each node represents a network-relevant component such as a router, switch, server, or virtualized network element. A_{enc} contains the physical neighborhood used by the representation encoder. For RCA, $W^{(t)}$ is constructed from the forwarding relations recorded for the corresponding fault event, with an edge directed from the potential fault source toward the downstream affected node. The experiments use unit edge weights because comparable capacity, edge-type, and delay metadata are not available for every event; physical links without a recorded direction are represented by reciprocal unit edges.

Component	Default setting/value
Input window length T	20 time steps
Sampling interval	30 s
Hidden dimension d_h	64
Temporal encoder	2-layer Transformer encoder
Number of attention heads	4
Structural encoder	2-layer topology-aware graph encoder
Graph aggregation type	Symmetrically normalized A_{enc}
RCA propagation matrix	Directed row-normalized $P^{(t)}$
Masking scheme	Point-wise Bernoulli masking; zero fill after normalization
Mask ratio	0.15
Prediction horizon	1 step ahead
Reconstruction loss	Mean absolute error (MAE)
Prediction loss	Mean absolute error (MAE)
Loss weights (λ_1, λ_2)	$(1.00, 1.00)$
Topology regularization weight λ_3	0.10; validation grid $\{0, 0.01, 0.05, 0.10, 0.20, 0.50\}$
Anomaly score balance α	0.50
Top-m system aggregation	$m = 3$
RCA candidate ranking	Threshold-based candidates followed by the remaining monitored nodes; complete ranking retained for evaluation, with the first five displayed
Propagation depth L_p	3 hops
Path decay parameter δ	0.70
RCA score weights (γ, η)	$(0.60, 0.40)$
Optimizer	Adam
Initial learning rate	1×10^{-3}
Batch size	64
Maximum epochs	50
Early stopping patience	8 epochs
Gradient clipping	Global norm of 5.0

Table 4. Default implementation settings of the proposed framework.

For each node, we collect eight KPI dimensions at a sampling interval of 30 seconds. These KPIs are CPU utilization, memory usage, inbound traffic, outbound traffic, round-trip time (RTT), packet loss rate, queue length, and interface error count. This KPI set covers both host-side and network-side behaviors and provides enough information for anomaly detection and diagnosis.

We use a sliding-window setting in all experiments. The window length is 20 time steps, which corresponds to 10 minutes, and the stride is one time step. The data are split into training, validation, and test sets with ratios of 60%, 15%, and 25%, respectively. Training contamination is defined at the window level as the fraction of training windows whose time spans overlap a recorded fault-injection interval. We first construct a clean pool from windows that do not overlap any recorded injection interval and a disjoint contamination pool from injected-fault windows that are not used for validation or testing. For the main experiments, normal training windows are replaced with abnormal windows sampled from this pool, stratified across the six fault categories, to obtain an exact contamination ratio of 3% while keeping the number of training windows fixed. Thus, the ratio is controlled from the fault-injection schedule rather than estimated from the detector's own predictions.

The 8%–15% abnormal-window ratio in the test set is an event-enriched evaluation design, not an estimate of the natural anomaly incidence in a production network. The controlled fault campaigns deliberately include enough events from each of the six fault categories and three topology scales to support per-category and RCA evaluation. Fault windows are retained together with their surrounding normal context, which produces a substantially higher prevalence than would normally be expected in continuous operation. Test labels are not used to select thresholds or model parameters. Nevertheless, Precision, F1-score, and AUPRC depend on class prevalence; consequently, their reported values characterize the controlled evaluation mixture and should not be interpreted as the expected alert precision at a production anomaly rate below 1%. AUROC is less directly affected by prevalence, but deployment at such low base rates would still require operating-point calibration and explicit false-alarm evaluation on longer production traces. Because the present controlled campaigns do not provide long-duration test streams with a natural anomaly prevalence below 1%, we do not report a sub-1% deployment evaluation.

To evaluate both anomaly detection and root cause node localization, we inject six representative fault types into the testbed: CPU overload, bandwidth bottleneck, packet loss, link delay increase, interface failure, and routing instability. These faults are common in network operation scenarios and produce both local KPI changes and multi-hop topology-level propagation effects. Each injected fault event contains exactly one deliberately injected fault at node r_e, which is treated as the sole ground-truth root cause. Other nodes that become anomalous during the same event, including downstream nodes affected through fault propagation, are labeled as affected nodes and are excluded from the ground-truth root-cause set. These labels are used only for evaluation.

The proposed framework uses a 2-layer Transformer encoder as the temporal module and a 2-layer topology-aware graph encoder as the structural module. The hidden dimension is set to 64 and the number of attention heads is 4. At every training epoch, 15% of the normalized time–node–KPI entries are independently masked and replaced by zero; reconstruction MAE is computed only at those locations. The model predicts one future step and reconstructs masked values. We use Adam as the optimizer with an initial learning rate of 1×10^{-3}. The batch size is 64. The maximum number of epochs is 50, and the global gradient norm is clipped at 5.0. Early stopping is used with a patience of 8 epochs.

To improve reproducibility, all experiments follow one fixed protocol. KPI normalization statistics are estimated from the training split only. Threshold τ is chosen on the validation split by maximizing the system-level window-based F1-score, whereas τ_n is chosen independently by maximizing post-alarm node-level A@1. The topology regularization weight λ_3, propagation decay δ, top-m aggregation size, and RCA coefficient γ are also selected only on validation data; $\eta = 1 - \gamma$. All selections are then fixed across the three topology scales and fault types in the test set. Every proposed-method and baseline experiment is evaluated using the same five aligned random seeds (17, 29, 43, 71, and 101). Tables 6, 7 and 8 report the cross-seed mean and sample standard deviation. Two-sided 95% Student-t confidence intervals are calculated as $\bar{x} \pm t_{0.975,4} s/\sqrt{5}$.

Two paired comparisons are specified before examining the test results: the F1-score of the proposed detector versus DCdetector, and the A@1 of the proposed RCA method versus the restricted-input Nezha adaptation. The replicate unit is the aligned random seed, so each test uses five paired differences. We use a two-sided paired Student's t-test and report the mean paired difference, its 95% confidence interval, $t(4)$, the two-sided p-value, and Cohen's d_z (the mean paired difference divided by the sample standard deviation of the paired differences). Holm adjustment is applied across these two prespecified comparisons. Statistical significance is assessed at a two-sided adjusted $p < 0.05$, but interpretation emphasizes the paired effect estimate and confidence interval because only five seed pairs are available. For RCA evaluation, candidate rankings are generated after the system-level alarm is triggered. Aggregate metrics are first computed within each topology scale and seed, then macro-averaged across the 12-, 24-, and 48-node settings; the scale-specific results are reported separately in Tables 10 and 11.

Table 5 summarizes the main experimental settings.

Public external benchmark. For external validation beyond the controlled Mininet environment, we use Dataset D from the public DejaVu benchmark[30]. Dataset D is collected from the open-source Train-Ticket microservice benchmark and contains 156 injected failure events, 5,724 monitored metrics, 1,044 failure units, and 23 failure classes. The released package provides time-series metrics, failure dependency graphs, and event-level root-cause labels. We map failure units to monitored nodes, use the released dependency graph as the directed topology prior, and retain only metric and graph inputs so that the observability budget remains compatible with the proposed framework. In our external evaluation protocol, the failure events were randomly divided into training, validation, and test sets at ratios of 40%, 20%, and 40%, respectively, and the evaluation was repeated over five random splits.

Item	Configuration
Topology scales	Small/Medium/Large
Number of monitored nodes	12/24/48
Number of KPI dimensions per node	8
Sampling interval	30 s
Window length	20 time steps
Training split	60%
Validation split	15%
Test split	25%
Training anomaly contamination	3% abnormal windows, controlled from recorded injection intervals
Controlled test anomaly ratio	8%–15% (event-enriched evaluation)
Fault types	6 categories
RCA labels	One injected root node per event; affected propagation nodes recorded separately and excluded from root-cause accuracy

Table 5. Experimental data summary.

Baselines and evaluation metrics

We compare the proposed framework with two groups of baselines. The first group is used for anomaly detection. The second group is used for root cause node localization.

The anomaly detection baselines include ordinary least squares (OLS), a first-order vector autoregressive model (VAR(1)), OmniAnomaly[1], MSCRED[2], USAD[3], THOC[4], MTAD-GAT[5], GDN[6], InterFusion[7], Anomaly Transformer[8], TranAD[9], and DCdetector[10]. OLS predicts each KPI from its lagged values, whereas VAR(1) jointly predicts all KPIs from the immediately preceding multivariate observation. Their anomaly scores are the normalized absolute prediction residuals. These two models provide simple but strong linear reference points alongside reconstruction-based, forecasting-based, graph-based, and Transformer-based approaches.

The RCA baselines include OLS-residual and VAR(1)-residual ranking. The remaining baselines are CauseInfer[11], MonitorRank[12], CloudRanger[13], MicroCause[14], TraceRCA[15], and Nezha[16]. The two linear baselines rank nodes by their maximum normalized KPI prediction residual after an alarm. The remaining methods represent ranking-based or propagation-aware diagnosis methods. TraceRCA is originally trace-based, while Nezha uses multimodal observability data; their implementations in this evaluation are therefore explicitly treated as restricted-input adaptations rather than reproductions of their native full-observability settings.

For anomaly detection, all baselines are retrained on the same training split, use the same KPI windows, and receive the same normalized KPI inputs. Detection thresholds are selected on the validation split using each method's anomaly score output. For methods that do not explicitly model topology, only KPI inputs are provided; topology-aware methods receive the same encoder adjacency. For RCA, we report a controlled input-matched comparison: every method receives the same post-alarm candidate nodes, node-level KPI statistics, event timestamps, and monitored topology. This protocol isolates differences among ranking mechanisms under a common observability budget, but it does not measure the performance ceiling of methods designed for traces, logs, or other richer inputs. Accordingly, the TraceRCA and Nezha rows in Table 8 are labeled as restricted-input adaptations, and we do not claim superiority over their original native-observability systems. Appendix Tables 20 and 21 report the shared training and model-selection protocol and the implementation-specific settings of all 12 detection and eight RCA baselines, respectively.

For anomaly detection, we report Precision, Recall, F1-score, AUROC, AUPRC, and average detection delay. F1-score is the main metric because it balances false alarms and missed detections. AUROC and AUPRC evaluate ranking quality under different thresholds. Detection delay measures how quickly a method can identify an abnormal event after it starts. All anomaly detection metrics are computed at the window level and are macro-averaged over the three topology scales.

For root cause node localization, we report localization accuracy at rank cutoffs 1, 3, and 5, denoted A@1, A@3, and A@5, respectively, together with mean reciprocal rank (MRR) and Mean Rank, following ranking-based RCA evaluation practice[13,15]. For event e, let q_e denote the position of its sole injected root node r_e in the complete predicted ranking of candidate nodes. The metrics are

$$\text{A@}k = \frac{1}{E}\sum_{e=1}^{E}\mathbb{I}(q_e \le k), \qquad \text{MRR} = \frac{1}{E}\sum_{e=1}^{E}\frac{1}{q_e}, \qquad \text{Mean Rank} = \frac{1}{E}\sum_{e=1}^{E} q_e. \tag{42}$$

The ranking is not truncated when MRR and Mean Rank are calculated; the cutoffs are used only to define A@1, A@3, and A@5. Downstream affected nodes are excluded from the ground-truth root-cause set and from all five localization metrics. Higher A@k and MRR are better, whereas lower Mean Rank is better. RCA metrics are computed over fault events after alarm triggering, first within each topology scale and seed and then by macro-averaging across fault types and topology scales.

For diagnostic evidence, we use two practical measures. The first is KPI contribution hit rate, which checks whether the KPI with the largest residual share matches the dominant KPI change of the injected fault. The second is path consistency rate, which checks whether the extracted directed explanation path overlaps with

Method	Precision	Recall	F1-score	AUROC	AUPRC
OLS	0.8339 ± 0.0078	0.8282 ± 0.0129	0.8308 ± 0.0038	0.9115 ± 0.0043	0.9141 ± 0.0059
VAR(1)	0.8598 ± 0.0113	0.8436 ± 0.0117	0.8514 ± 0.0051	0.9295 ± 0.0061	0.9301 ± 0.0057
OmniAnomaly[1]	0.8757 ± 0.0073	0.8909 ± 0.0158	0.8831 ± 0.0071	0.9545 ± 0.0023	0.9562 ± 0.0019
MSCRED[2]	0.8757 ± 0.0086	0.8818 ± 0.0162	0.8785 ± 0.0065	0.9486 ± 0.0046	0.9493 ± 0.0033
USAD[3]	0.8717 ± 0.0159	0.8760 ± 0.0185	0.8736 ± 0.0120	0.9470 ± 0.0065	0.9486 ± 0.0066
THOC[4]	0.8880 ± 0.0180	0.8934 ± 0.0099	0.8904 ± 0.0070	0.9576 ± 0.0041	0.9591 ± 0.0050
MTAD-GAT[5]	0.8975 ± 0.0110	0.8913 ± 0.0059	0.8942 ± 0.0048	0.9619 ± 0.0051	0.9623 ± 0.0056
GDN[6]	0.8989 ± 0.0110	0.8991 ± 0.0122	0.8988 ± 0.0084	0.9646 ± 0.0051	0.9653 ± 0.0057
InterFusion[7]	0.9032 ± 0.0112	0.8951 ± 0.0077	0.8989 ± 0.0048	0.9653 ± 0.0034	0.9663 ± 0.0030
Anomaly Transformer[8]	0.9087 ± 0.0061	0.9051 ± 0.0076	0.9068 ± 0.0059	0.9698 ± 0.0050	0.9711 ± 0.0047
TranAD[9]	0.9136 ± 0.0100	0.9120 ± 0.0084	0.9127 ± 0.0053	0.9705 ± 0.0028	0.9712 ± 0.0031
DCdetector[10]	0.9209 ± 0.0102	0.9178 ± 0.0069	0.9192 ± 0.0045	0.9753 ± 0.0011	0.9758 ± 0.0012
Proposed	**0.9230 ± 0.0052**	**0.9240 ± 0.0074**	**0.9235 ± 0.0037**	**0.9780 ± 0.0019**	**0.9789 ± 0.0019**

Table 6. Five-seed anomaly detection results on the network operation testbed. Values are mean ± sample standard deviation ($n = 5$). Significant values are in bold.

Method	Mean ± SD	95% CI
OLS	4.898 ± 0.010	[4.885, 4.910]
VAR(1)	4.299 ± 0.012	[4.284, 4.314]
OmniAnomaly[1]	4.102 ± 0.009	[4.091, 4.113]
MSCRED[2]	N/A	N/A
USAD[3]	N/A	N/A
THOC[4]	3.693 ± 0.013	[3.677, 3.709]
MTAD-GAT[5]	3.297 ± 0.012	[3.283, 3.311]
GDN[6]	3.099 ± 0.013	[3.082, 3.116]
InterFusion[7]	N/A	N/A
Anomaly Transformer[8]	2.996 ± 0.009	[2.984, 3.007]
TranAD[9]	2.799 ± 0.007	[2.790, 2.808]
DCdetector[10]	2.697 ± 0.014	[2.680, 2.714]
Proposed	**2.400 ± 0.010**	**[2.387, 2.413]**

Table 7. Five-seed anomaly detection delay in time steps. Confidence intervals use $t_{0.975,4} = 2.776$. N/A applies only to MSCRED, USAD, and InterFusion. Their retained outputs do not contain window-level alarm trajectories aligned with recorded fault onset, so a comparable event-level delay cannot be reconstructed. We do not estimate, interpolate, or impute delay from aggregate detection metrics

the recorded propagation path. These are task-specific evidence measures and should not be interpreted as validation of causal feature attribution.

Main results

We first report the anomaly detection results. Table 6 reports the five-seed mean and sample standard deviation for every comparator. The proposed framework obtains an F1-score of 0.9235 ± 0.0037, compared with 0.9192 ± 0.0045 for DCdetector. The corresponding scale-stratified results are reported in Table 10.

The mean F1-score is 0.43 percentage points higher than that of DCdetector. However, the aligned five-seed difference is not statistically significant ($\Delta F1 = 0.00428$, 95% CI $[-0.00419, 0.01275]$, paired $t(4) = 1.403$, Holm-adjusted $p = 0.233$; Table 9). We therefore interpret this result as a higher mean under the evaluated runs rather than evidence of a statistically established improvement over DCdetector. The reconstruction branch captures local deviations, the prediction branch responds to short-term future deviations, and the topology-aware encoder supplies structural context that may help separate topology-consistent anomalies from isolated KPI noise.

Table 7 reports the five-seed mean, sample standard deviation, and 95% confidence interval of detection delay. The proposed method has a mean delay of 2.400 ± 0.010 time steps, compared with 2.697 ± 0.014 for DCdetector. Delay is not reported for MSCRED, USAD, or InterFusion because their retained evaluation outputs contain only event-level aggregate scores, not the window-level alarm trajectories aligned to recorded fault onset that are required by the common onset-to-first-alarm definition.

Method	A@1 ↑	A@3 ↑	A@5 ↑	MRR ↑	Mean Rank ↓
OLS-residual	0.2804 ± 0.0113	0.4384 ± 0.0183	0.5524 ± 0.0171	0.4265 ± 0.0125	5.1322 ± 0.1583
VAR(1)-residual	0.3520 ± 0.0090	0.5091 ± 0.0072	0.6180 ± 0.0091	0.4882 ± 0.0076	4.6202 ± 0.0494
CauseInfer[11]	0.4391 ± 0.0232	0.5927 ± 0.0263	0.6964 ± 0.0214	0.5632 ± 0.0209	4.0089 ± 0.1982
MonitorRank[12]	0.4947 ± 0.0211	0.6296 ± 0.0173	0.7324 ± 0.0116	0.6061 ± 0.0149	3.6720 ± 0.1043
CloudRanger[13]	0.5904 ± 0.0301	0.7291 ± 0.0261	0.8211 ± 0.0297	0.6903 ± 0.0257	2.9660 ± 0.2514
MicroCause[14]	0.6169 ± 0.0314	0.7516 ± 0.0284	0.8382 ± 0.0313	0.7122 ± 0.0264	2.8116 ± 0.2325
TraceRCA (restricted)[15]	0.6736 ± 0.0154	0.7962 ± 0.0165	0.8860 ± 0.0151	0.7586 ± 0.0125	2.4182 ± 0.1081
Nezha (restricted)[16]	0.7067 ± 0.0229	0.8229 ± 0.0219	0.9118 ± 0.0156	0.7854 ± 0.0190	2.2109 ± 0.1412
Proposed	**0.7613 ± 0.0163**	**0.8753 ± 0.0154**	**0.9556 ± 0.0044**	**0.8330 ± 0.0124**	**1.8198 ± 0.0646**

Table 8. Five-seed root cause node localization results under one injected ground-truth root per event. Values are mean ± sample standard deviation ($n = 5$); affected propagation nodes are excluded from all metrics. Significant values are in bold.

Comparison	Metric	Mean paired difference	95% CI	$t(4)$	Cohen's d_z	Raw p	Holm-adjusted p
Proposed – DCdetector	F1-score	0.00428	[−0.00419, 0.01275]	1.403	0.628	0.23315	0.23315
Proposed – Nezha (restricted)	A@1	0.05467	[0.02538, 0.08395]	5.182	2.318	0.00660	0.01319

Table 9. Prespecified two-sided paired t-tests across the five aligned random seeds. Holm-adjusted p-values control the family of two comparisons.

We next report the RCA results. Table 8 reports the five-seed mean and sample standard deviation for every RCA comparator under the single-injected-root definition. The proposed method obtains an A@1 of 0.7613 ± 0.0163, an MRR of 0.8330 ± 0.0124, and a Mean Rank of 1.8198 ± 0.0646. Table 11 reports the corresponding scale-stratified results.

Compared with the restricted-input Nezha adaptation, the proposed method has a 5.47-percentage-point higher mean A@1. The aligned five-seed difference is statistically significant (ΔA@1 = 0.05467, 95% CI [0.02538, 0.08395], paired $t(4) = 5.182$, Holm-adjusted $p = 0.013$; Table 9). This comparison is restricted to the controlled KPI-plus-topology input adaptation and does not represent Nezha operating on its native multimodal inputs. The normalized node-level anomaly evidence identifies suspicious candidates, and directed propagation consistency distinguishes potential upstream sources from downstream affected nodes. Residual-based KPI contributions are displayed only after ranking and therefore cannot improve or bias the reported RCA performance.

The scale-specific ordering is not identical across topology sizes. The proposed detector has a higher mean F1 than DCdetector at 12 nodes (0.9410 versus 0.9343) and 24 nodes (0.9256 versus 0.9126), whereas DCdetector has the higher mean at 48 nodes (0.9107 versus 0.9038). Thus, the aggregate 0.43-percentage-point F1 difference should not be interpreted as uniform superiority across scale. The proposed method's mean detection delay increases from 2.2020 ± 0.0075 to 2.3890 ± 0.0235 and 2.6094 ± 0.0098 time steps. For RCA, the proposed method retains the highest mean A@1 at all three scales, but A@1 decreases from 0.7947 ± 0.0310 to 0.7600 ± 0.0220 and 0.7293 ± 0.0092. Its MRR decreases from 0.8633 ± 0.0224 to 0.8321 ± 0.0164 and 0.8036 ± 0.0041, while Mean Rank increases from 1.5320 ± 0.1033 to 1.7680 ± 0.1295 and 2.1593 ± 0.0341. These results show increasing difficulty with topology size while directly exposing the scale-dependent changes in method ordering (Tables 10, 11).

External benchmark validation

Table 12 presents the cross-system evaluation on the public Train-Ticket benchmark. OLS and VAR(1) use prediction residuals for both detection and node ranking. CloudRanger and DejaVu are evaluated only for RCA because they assume that a failure event has already been identified. DejaVu is run in its native metric-plus-failure-dependency-graph setting, rather than being restricted from a richer observability space.

Across five splits, the proposed method obtains an external detection F1-score of 0.846 ± 0.007 (95% CI [0.837, 0.855]) and an A@1 of 0.742 ± 0.011 (95% CI [0.728, 0.756]). Relative to native DejaVu, A@1 increases by 2.4 percentage points, A@3 increases by 1.7 percentage points, and MAR decreases from 5.03 to 4.31, corresponding to a 14.3% reduction. Relative to VAR(1), the F1-score increases by 7.7 percentage points. These results show that the performance gains observed in the controlled testbed persist under the public benchmark's substantially larger candidate space and different system structure.

Ablation and sensitivity analysis

We use controlled secondary analyses to examine the contribution of each main component and the directed propagation design. These analyses compare removal of the topology encoder, removal of the prediction branch, replacement of the directed propagation matrix by a static undirected adjacency matrix, and removal of propagation consistency. They use a fixed secondary-analysis protocol and are therefore reported as within-

Method	12 nodes	24 nodes	48 nodes
OLS	0.8457 ± 0.0162	0.8337 ± 0.0120	0.8131 ± 0.0147
VAR(1)	0.8657 ± 0.0104	0.8516 ± 0.0172	0.8369 ± 0.0181
OmniAnomaly[1]	0.8940 ± 0.0078	0.8857 ± 0.0138	0.8696 ± 0.0100
MSCRED[2]	0.9000 ± 0.0151	0.8721 ± 0.0089	0.8632 ± 0.0081
USAD[3]	0.8973 ± 0.0080	0.8730 ± 0.0147	0.8507 ± 0.0199
THOC[4]	0.8978 ± 0.0109	0.8922 ± 0.0061	0.8813 ± 0.0124
MTAD-GAT[5]	0.9129 ± 0.0114	0.8933 ± 0.0106	0.8765 ± 0.0126
GDN[6]	0.9157 ± 0.0113	0.8956 ± 0.0150	0.8852 ± 0.0068
InterFusion[7]	0.9139 ± 0.0119	0.8975 ± 0.0120	0.8855 ± 0.0188
Anomaly Transformer[8]	0.9190 ± 0.0048	0.9190 ± 0.0066	0.8824 ± 0.0159
TranAD[9]	0.9257 ± 0.0072	0.9129 ± 0.0096	0.8995 ± 0.0060
DCdetector[10]	0.9343 ± 0.0114	0.9126 ± 0.0059	**0.9107 ± 0.0145**
Proposed	**0.9410 ± 0.0111**	**0.9256 ± 0.0088**	0.9038 ± 0.0090

Table 10. Topology-scale-stratified F1-scores for all anomaly-detection methods. Values are five-seed mean ± sample standard deviation. Significant values are in bold.

Method	12 nodes	24 nodes	48 nodes
OLS-residual	0.3327 ± 0.0265	0.2680 ± 0.0358	0.2407 ± 0.0196
VAR(1)-residual	0.4300 ± 0.0139	0.3413 ± 0.0069	0.2847 ± 0.0126
CauseInfer[11]	0.5020 ± 0.0247	0.4240 ± 0.0344	0.3913 ± 0.0247
MonitorRank[12]	0.5440 ± 0.0166	0.4847 ± 0.0338	0.4553 ± 0.0458
CloudRanger[13]	0.6287 ± 0.0394	0.6087 ± 0.0348	0.5340 ± 0.0567
MicroCause[14]	0.6780 ± 0.0229	0.6094 ± 0.0292	0.5633 ± 0.0523
TraceRCA (restricted)[15]	0.7373 ± 0.0153	0.6753 ± 0.0269	0.6080 ± 0.0331
Nezha (restricted)[16]	0.7540 ± 0.0312	0.7027 ± 0.0224	0.6633 ± 0.0317
Proposed	**0.7947 ± 0.0310**	**0.7600 ± 0.0220**	**0.7293 ± 0.0092**

Table 11. Topology-scale-stratified A@1 for all root-cause-localization methods under the single-root definition. Values are five-seed mean ± sample standard deviation. Significant values are in bold.

Method	Detection F1	A@1	A@3	A@5	MAR
OLS-residual	0.731	0.344	0.529	0.641	21.60
VAR(1)-residual	0.769	0.397	0.596	0.699	17.20
CloudRanger[13]	–	0.512	0.711	0.786	11.40
DejaVu[30]	–	0.718	0.884	0.929	5.03
Proposed	**0.846**	**0.742**	**0.901**	**0.944**	**4.31**

Table 12. External validation on the public Train-Ticket benchmark. Significant values are in bold.

protocol changes from the selected reference configuration. This presentation avoids treating their absolute scores as additional estimates of the three-scale, five-seed main result in Tables 6 and 8. There is no "without attribution" ranking ablation because residual-based KPI contributions do not enter the RCA score.

Table 13 shows that all tested components contribute under the controlled ablation protocol. Removing the topology encoder causes the largest degradation, changing F1-score by −0.016 and A@1 by −0.070 relative to the reference configuration. Removing the prediction branch changes F1-score by −0.012, whereas removing propagation consistency changes A@1 by −0.048.

Replacing the directed propagation matrix with static undirected proximity leaves detection performance unchanged but changes A@1 by −0.016 and A@3 by −0.011. Removing propagation consistency is equivalent to the endpoint $(\gamma, \eta) = (1, 0)$ and changes A@1 by −0.048 and A@3 by −0.031. Thus, within this controlled ablation protocol, adding topology-propagation consistency to local anomaly intensity contributes 4.8 percentage points in A@1 and 3.1 percentage points in A@3. These results support both the use of propagation evidence and its directionality in the RCA stage. All variants use the same secondary-analysis split, threshold-selection procedure, and fixed ranking hyperparameters.

Variant	ΔF1-score	ΔAUROC	ΔA@1	ΔA@3
w/o Topology	−0.016	−0.013	−0.070	−0.055
w/o Prediction	−0.012	−0.008	−0.023	−0.015
Static undirected propagation	0.000	0.000	−0.016	−0.011
w/o Propagation	0.000	0.000	−0.048	−0.031
Selected reference configuration	**0.000**	**0.000**	**0.000**	**0.000**

Table 13. Within-protocol ablation changes relative to the selected reference configuration. Values are absolute metric differences; zero denotes the reference configuration. Significant values are in bold.

Setting	Value	ΔF1-score	ΔA@1
Window length T	10	−0.017	−0.041
Window length T	20	**0.000**	**0.000**
Window length T	30	−0.004	−0.010
Mask ratio	0.10	−0.005	−0.012
Mask ratio	0.15	**0.000**	**0.000**
Mask ratio	0.20	−0.003	−0.007
Propagation depth L_p	2	0.000	−0.015
Propagation depth L_p	3	**0.000**	**0.000**
Propagation depth L_p	4	−0.001	−0.006
Decay δ	0.3	0.000	−0.013
Decay δ	0.5	0.000	−0.005
Decay δ	0.7	**0.000**	**0.000**
Decay δ	0.9	−0.001	−0.008

Table 14. Within-protocol sensitivity changes relative to the validation-selected setting for each parameter group. Values are absolute metric differences.

γ	$\eta = 1 - \gamma$	ΔA@1	ΔA@3
0.0	1.0	−0.033	−0.022
0.1	0.9	−0.026	−0.018
0.2	0.8	−0.018	−0.013
0.3	0.7	−0.011	−0.008
0.4	0.6	−0.005	−0.004
0.5	0.5	−0.002	−0.001
0.6	0.4	**0.000**	**0.000**
0.7	0.3	−0.003	−0.003
0.8	0.2	−0.010	−0.008
0.9	0.1	−0.023	−0.018
1.0	0.0	−0.048	−0.031

Table 15. Within-protocol changes in root-cause node localization relative to the validation-selected RCA score weights $(\gamma, \eta) = (0.6, 0.4)$.

We further study sensitivity to window length T, point-mask ratio, propagation depth L_p, and the single hop-decay parameter δ. Each setting varies one parameter while holding the remaining secondary-analysis protocol fixed. Table 14 reports changes relative to the validation-selected configuration ($T = 20$, mask ratio 0.15, $L_p = 3$, and $\delta = 0.7$). The largest A@1 change is −0.041 across window lengths, −0.012 across mask ratios, −0.015 across propagation depths, and −0.013 across the tested decay values. The decreases at $\delta = 0.3$ and $\delta = 0.9$ are consistent with loss of relevant multi-hop evidence under aggressive decay and inclusion of less relevant distant nodes under weak decay, respectively. These within-protocol contrasts support $\delta = 0.7$ as a shared operating value without claiming separate optimality for every fault category.

To examine the RCA mixing coefficient, we vary γ from 0 to 1 in increments of 0.1 and set $\eta = 1 - \gamma$. Table 15 shows a broad stable region around the validation-selected setting $(\gamma, \eta) = (0.60, 0.40)$ rather than an isolated optimum. Relative to that setting, A@1 changes by no more than −0.010 for $\gamma \in [0.4, 0.8]$. The endpoint $\gamma = 0$ uses only propagation consistency and changes A@1/A@3 by $-0.033/-0.022$. The endpoint $\gamma = 1$ uses only

Training contamination ratio	ΔF1-score	ΔAUROC	ΔA@1
0%	+0.004	+0.004	+0.004
1%	+0.002	+0.002	+0.002
3%	**0.000**	**0.000**	**0.000**
5%	−0.007	−0.006	−0.008
10%	−0.024	−0.018	−0.025
15%	−0.049	−0.037	−0.050

Table 16. Within-protocol changes relative to the 3% abnormal-window contamination reference condition. Values are absolute metric differences.

Quantity	Five-seed summary	95% CI
Completed runs without non-finite loss	5/5	–
Early-stopping epoch	32.4 ± 3.1	[28.6, 36.2]
F1-score	0.9235 ± 0.0037	[0.9188, 0.9281]
A@1	0.7613 ± 0.0163	[0.7411, 0.7816]

Table 17. Five-seed training stability of the proposed objective.

local anomaly intensity and changes the two metrics by −0.048/ − 0.031. The final coefficient is selected on the validation split and fixed before test evaluation.

We also test whether performance degrades when the predominantly normal training assumption is progressively violated. Starting from the clean training pool, we construct training sets with abnormal-window contamination ratios of 0%, 1%, 3%, 5%, 10%, and 15%. For each nonzero condition, normal windows are replaced by abnormal windows sampled from the disjoint contamination pool with the six fault categories represented proportionally. The total training-set size, validation and test sets, model configuration, and model-selection protocol remain unchanged within this secondary analysis.

Table 16 reports changes relative to the 3% reference condition and shows a gradual rather than abrupt degradation. Increasing contamination to 5% changes F1 and A@1 by −0.007 and −0.008, respectively. At 10%, the corresponding changes are −0.024 and −0.025, whereas at 15% they reach −0.049 and −0.050. The small changes between 0%, 1%, and 3% indicate tolerance to limited contamination. The consistent decline beyond 5% confirms that the method is not insensitive to severely contaminated training data. These within-protocol contrasts delimit the tested robustness range rather than guaranteeing unchanged performance at arbitrary contamination levels.

Training stability is assessed over the same five random seeds. As summarized in Table 17, all five runs complete without non-finite losses. Training stops after 32.4 ± 3.1 epochs on average, with a 95% confidence interval of [28.6, 36.2]. The mean F1-score is 0.9235 ± 0.0037 (95% CI [0.9188, 0.9281]), and the mean A@1 is 0.7613 ± 0.0163 (95% CI [0.7411, 0.7816]). These intervals quantify cross-seed uncertainty and indicate that the proposed objective does not exhibit substantial seed-dependent instability.

Explainability analysis and case study

We finally examine the diagnostic evidence produced by the framework. The output includes two parts: residual-based KPI contributions and a directed topology-level path.

At the KPI level, the framework highlights the metrics with the largest shares of the reconstruction and prediction residuals. Under CPU-overload faults, CPU utilization, queue length, and interface error count receive the highest contribution scores. Under link-delay faults, RTT, packet loss, and outbound traffic imbalance are the dominant KPI contributions. These residual shares help operators inspect the observed anomaly but do not establish a causal attribution of the model decision.

At the topology level, the framework extracts a propagation subgraph rooted at the top-ranked candidate node. This subgraph shows how the abnormal behavior may travel from the source to downstream nodes. This form of explanation is useful because it gives operators a concrete path to inspect instead of only a ranking score.

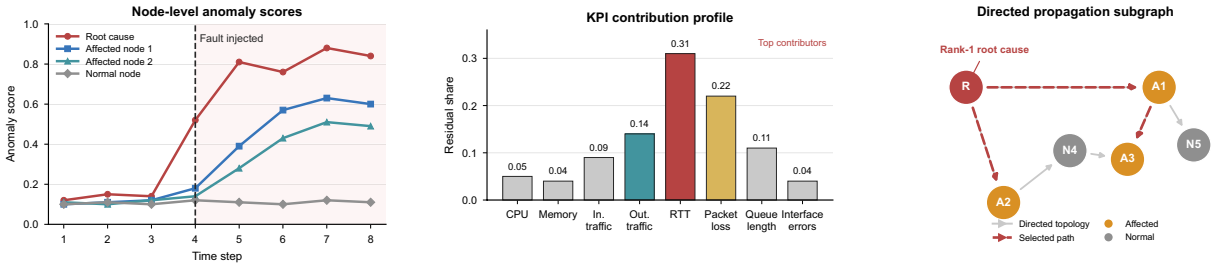
Table 18 summarizes the explainability results.

The KPI contribution hit rate reaches 0.809 at rank 1 and 0.919 at rank 3, indicating that the dominant injected-fault KPI is usually among the largest residual contributors. The directed explanation subgraph achieves a path consistency rate of 0.874, showing that most extracted paths overlap with the recorded propagation paths. These measures evaluate the operational usefulness of the evidence output; they do not convert residual shares into causal feature attributions.

Figure 3 presents a representative fault case, including the node-level anomaly score curves, the largest residual-based KPI contributions of the root cause node, and the extracted propagation subgraph.

Metric	Value	Description
KPI contribution hit rate@1	0.809	Largest residual share matches the dominant fault KPI
KPI contribution hit rate@3	0.919	A top-3 residual share matches the dominant fault KPI
Path consistency rate	0.874	Directed explanation path overlaps the recorded propagation path

Table 18. Explainability results.



(a) Node-level anomaly score curves. (b) Largest residual-based KPI contributions. (c) Extracted topology-aware propagation subgraph.

Fig. 3. Case study of a representative network fault event. (**a**) Node-level anomaly scores show that the true root cause node becomes abnormal earlier and more strongly than downstream affected nodes. (**b**) Residual-based KPI contributions identify the metrics with the largest observed reconstruction/prediction errors at the root cause node. (**c**) The extracted directed propagation subgraph highlights a topology-consistent path from the root cause to affected nodes.

Nodes	Parameters (M)	Training time/epoch (s)	Detection latency (ms)	RCA latency (ms)	Peak memory (GB)
12	0.42	4.8	3.1	0.8	0.68
24	0.42	8.9	5.4	1.4	0.82
48	0.42	17.1	9.7	2.7	1.09

Table 19. Runtime and resource cost at different topology scales.

Runtime and resource analysis

We measure computational efficiency with batch size one after 100 warm-up windows and 1,000 timed windows. Detection latency covers normalization and the forward pass for one KPI window; RCA latency covers candidate construction, directed propagation scoring, and explanation-path extraction for one alarm. Peak memory is recorded during online inference. Table 19 reports the results obtained with a single NVIDIA RTX 3090 GPU and an Intel Core i9-10940X CPU. These measurements characterize the evaluated implementation and hardware rather than production deployability under arbitrary network conditions.

Under the 48-node setting, the complete online path requires 9.7 ms for detection and 2.7 ms for RCA, yielding a combined latency of 12.4 ms after an input window is available. Peak inference memory increases from 0.68 GB at 12 nodes to 1.09 GB at 48 nodes, while the parameter count remains 0.42 M because model weights are shared across nodes. The measured computation time is small relative to the 30-s KPI sampling interval and supports computational feasibility under the evaluated hardware and topology scales; production deployment would require system-level validation under the target workload and monitoring infrastructure.

Conclusion and discussion

This paper proposed a topology-aware, self-supervised AIOps framework for multivariate KPI anomaly detection and root cause node localization in network operations. The framework learns normal patterns from unlabeled KPI streams, detects anomalies at the KPI, node, and system levels, and then ranks candidate root cause nodes by combining normalized anomaly intensity with directed topology-aware propagation consistency.

The main value of the proposed method lies in three aspects. First, it reduces the dependence on labeled fault data by using self-supervised learning. Second, it introduces a directed propagation prior into the RCA

stage. Third, it reports descriptive residual-based KPI contributions and topology-level propagation paths that operators can inspect alongside the ranked nodes.

Across five random seeds, the proposed framework achieves an F1-score of 0.9235 ± 0.0037 for anomaly detection and an A@1 of 0.7613 ± 0.0163 for root-cause node localization on the Mininet testbed. Its mean F1-score is 0.43 percentage points higher than that of DCdetector, but the paired difference is not statistically significant (Holm-adjusted $p = 0.233$). Under the controlled KPI-plus-topology observability budget, A@1 is 5.47 percentage points higher than that of the restricted-input Nezha adaptation (Holm-adjusted $p = 0.013$); this comparison does not establish superiority over Nezha operating with its native multimodal inputs. The corresponding 95% confidence intervals are [0.9188, 0.9281] for F1 and [0.7411, 0.7816] for A@1. The residual-contribution analysis obtains hit rates of 0.809 at rank 1 and 0.919 at rank 3, while the directed explanation paths achieve a consistency rate of 0.874. On the public Train-Ticket benchmark, the framework achieves an F1-score of 0.846 and an A@1 of 0.742, with corresponding 95% confidence intervals of [0.837, 0.855] and [0.728, 0.756]. The complete online path requires 12.4 ms in the 48-node setting. Collectively, these results support cross-system transferability and computational feasibility under the evaluated datasets, topology scales, hardware, and observability settings; they do not establish production-level generalizability or deployability. The residual shares remain descriptive rather than causal explanations.

This study still has some limitations. The Train-Ticket evaluation tests transfer to a public semi-real microservice environment, but it cannot substitute for prospective deployment in a live production network. The Mininet test sets are event-enriched to ensure coverage of all fault categories and topology scales; their 8%–15% abnormal-window prevalence is therefore substantially higher than the base rate expected in continuous production monitoring. Because Precision, F1-score, and AUPRC depend on prevalence, the corresponding values should not be extrapolated directly to deployments with anomaly rates below 1%. Such deployment requires evaluation on longer-duration traces and threshold calibration at the required false-alarm budget. The RCA comparison on Mininet uses an equal KPI-plus-topology observability budget; consequently, the restricted TraceRCA and Nezha adaptations do not represent the performance of their original trace-based or multimodal systems. The framework also assumes that the topology prior is available and reasonably accurate. Although the formulation accepts a time-indexed directed weighted matrix, the present evaluation uses event-level forwarding snapshots with unit weights and therefore does not validate continuous routing updates, heterogeneous edge semantics, or propagation-delay calibration. Performance may decrease when topology snapshots are incomplete or stale. Moreover, the topology regularizer uses adjacency as a homogeneous similarity prior. Although the local residual input and the reconstruction and prediction objectives preserve node-specific information, the present experiments do not isolate performance as a function of node-role heterogeneity. The regularizer may therefore oversmooth legitimately different representations of adjacent routers, switches, servers, or virtualized components. Type-compatibility gates or relation-specific projections should be considered when reliable node and edge metadata are available. In addition, the residual-based KPI contribution is descriptive anomaly evidence rather than causal or gradient-based feature attribution, and the present design does not include other observability sources such as logs, traces, and configuration events.

In future work, we will extend the framework in five directions. First, we will study online adaptation under dynamic network conditions. Second, we will develop role- and relation-aware graph encoders and topology regularizers for heterogeneous nodes and edges. Third, we will integrate more heterogeneous observability signals for stronger diagnosis capability. Fourth, we will add a separately evaluated fault-type diagnosis module, for example a supervised or weakly supervised classifier that predicts CPU, memory, link, or routing fault categories after a source node has been localized. Fifth, we will explore more advanced explanation strategies to further improve the usability of AIOps systems in practice.

Data availability

The public DejaVu benchmark, including the Train-Ticket Dataset D metrics, failure dependency graphs, and root-cause labels, is available from Zenodo at https://doi.org/10.5281/zenodo.6955909. The Mininet testbed data and experiment configuration used in this study are available from the corresponding author on reasonable request.

Code availability

Code and reproducibility materials for the proposed computational pipeline are archived in Zenodo at https://doi.org/10.5281/zenodo.21990560. The archive includes the self-supervised anomaly detection model, topology-aware root cause node localization algorithm, multi-granularity anomaly scoring and diagnostic modules, baseline implementations, experiment configurations, evaluation scripts, reported result summaries, and automated tests. The Zenodo record is the persistent archived version cited for this manuscript.

Appendix: Baseline implementation and training details

This appendix reports the configurations used to place all baseline methods under the input and model-selection protocol described in section "Experiments". All methods use the same chronological train/validation/test separation, and no test labels are used for fitting, hyperparameter selection, early stopping, or threshold selection. TraceRCA and Nezha are evaluated as restricted-input adaptations because traces, logs, and other native multimodal signals are outside the common observability budget (Tables 20, 21).

Method	Input and anomaly score	Architecture, optimization, and model selection
Shared protocol	Train-only z-score-normalized KPI windows; length 20 and stride 1.	Chronological 60%/15%/25% train/validation/test split; five random seeds. Neural baselines use Adam, learning rate 10^{-3}, batch size 64, at most 50 epochs, early-stopping patience 8, and gradient clipping at global norm 5. The checkpoint and anomaly threshold are selected using validation data only.
OLS	Per-KPI one-step prediction; maximum normalized absolute residual forms the node score.	One lag, intercept included, fitted by ordinary least squares on the training split; no iterative optimization or regularization.
VAR(1)	Joint one-step multivariate prediction; maximum normalized absolute residual forms the node score.	Lag order 1 with intercept; coefficients fitted by multivariate least squares on the training split.
OmniAnomaly	Windowed KPI vector; negative reconstruction log-likelihood.	One GRU layer with hidden size 64; stochastic latent size 16. Monte Carlo anomaly score averaged over 10 samples. Common neural optimization protocol above.
MSCRED	Multi-scale KPI signature matrices; reconstruction error.	Signature windows $\{5, 10, 20\}$; three convolutional blocks with 32, 64, and 128 channels; ConvLSTM hidden channels matched to the corresponding blocks. Common neural optimization protocol above.
USAD	Flattened KPI window; weighted dual-decoder reconstruction error.	Encoder widths 64 and 32 with latent size 16; two symmetric decoders. The two reconstruction terms have equal weight at inference. Common neural optimization protocol above.
THOC	KPI window; temporal-clustering distance.	Recurrent hidden size 64, three temporal resolutions, and six temporal clusters. Cluster centers are initialized from training representations and updated jointly. Common neural optimization protocol above.
MTAD-GAT	KPI window; equal-weight forecast and reconstruction score.	Feature-oriented and temporal graph-attention blocks with four heads; GRU hidden size 64. Forecast and reconstruction scores are combined with weights 0.5/0.5. Common neural optimization protocol above.
GDN	KPI window and learned variable graph; one-step forecast residual.	Node embedding size 64; one graph-attention layer; each KPI node retains its top $\min(10, D-1)$ learned neighbors, where D is the number of KPI variables. Common neural optimization protocol above.
InterFusion	KPI window; hierarchical probabilistic reconstruction score.	Metric-level and temporal latent sizes 16; recurrent hidden size 64; 10 Monte Carlo samples at inference. Common neural optimization protocol above.
Anomaly Transformer	KPI window; association-discrepancy-weighted reconstruction score.	Three encoder layers, model dimension 64, four heads, and feed-forward dimension 256; dropout 0.1. The decision threshold is selected on the validation split rather than set from the test anomaly ratio. Common neural optimization protocol above.
TranAD	KPI window; two-phase reconstruction error.	Two Transformer encoder/decoder layers, model dimension 64, four heads, and feed-forward dimension 256; dropout 0.1. Common neural optimization protocol above.
DCdetector	KPI window; dual-attention contrastive discrepancy.	Three encoder layers, model dimension 64, four heads, and patch sizes $\{1, 5\}$; dropout 0.1. The anomaly threshold is selected on the validation split. Common neural optimization protocol above.

Table 20. Shared protocol and implementation settings for the anomaly-detection baselines.

Method	Common input and ranking evidence	Implementation and selection settings
Shared protocol	Alarm-triggered candidate nodes, node-level KPI statistics, event timestamps, and the monitored directed topology.	Nodes satisfying the validation-selected threshold are ranked first, followed by the remaining monitored nodes to retain a complete candidate ranking. The first five positions define A@1, A@3, and A@5 and are used for display; MRR and Mean Rank use the complete ranking. Rankings are produced only after a system alarm. Hyperparameters are selected on the validation split and then fixed across fault types and topology scales; results are averaged over five seeds.
OLS-residual	Maximum normalized per-node KPI residual from the OLS detector.	Candidates are sorted in descending residual order; ties are resolved by the mean residual across the eight KPIs.
VAR(1)-residual	Maximum normalized per-node KPI residual from VAR(1).	Candidates are sorted in descending residual order; ties are resolved by the mean residual across the eight KPIs.
CauseInfer	KPI anomaly series and topology-constrained causal influence.	Maximum lag 3; conditional-independence significance level 0.05. Candidate scores aggregate incoming causal influence from abnormal neighbors.
MonitorRank	Node anomaly evidence and monitored topology.	Personalized PageRank with anomaly scores as the restart prior; damping factor 0.85, tolerance 10^{-6}, and at most 100 iterations.
CloudRanger	KPI anomaly series and topology-filtered impact graph.	Maximum lag 3 and conditional-independence significance level 0.05; second-order random walk with restart probability 0.15, tolerance 10^{-6}, and at most 100 iterations.
MicroCause	Metric-level anomaly series and the monitored topology.	Maximum lag 3; causal-edge significance level 0.05. Metric-level causal scores are aggregated to nodes by maximum evidence and used to rank the common candidate set.
TraceRCA (restricted)	KPI anomaly evidence and monitored topology; trace spans and call-latency attributes are not supplied.	The topology graph replaces the trace graph. Edge evidence is the absolute lagged KPI correlation over the event window; maximum lag 3. Root scores use a restart walk with probability 0.15 and at most 100 iterations.
Nezha (restricted)	KPI windows and monitored topology; logs, traces, and event text are not supplied.	KPI encoder hidden size 64; two topology-attention layers with four heads and dropout 0.1. Adam, learning rate 10^{-3}, batch size 64, at most 50 epochs, patience 8, and gradient clipping at global norm 5.

Table 21. Shared protocol and implementation settings for the root-cause-analysis baselines.

Received: 8 May 2026; Accepted: 27 August 2026
Published online: 09 September 2026

References

1. Su, Y. et al. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2828–2837 (2019).

2. Zhang, C. et al. A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33, 1409–1416 (2019).
3. Audibert, J., Michiardi, P., Guyard, F., Marti, S. & Zuluaga, M. A. USAD: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 3395–3404 (2020).
4. Shen, L., Li, Z. & Kwok, J. T. Timeseries anomaly detection using temporal hierarchical one-class network. In *Advances in Neural Information Processing Systems*, Vol. 33, 13016–13026 (2020).
5. Zhao, H. Multivariate time-series anomaly detection via graph attention network. In *2020 IEEE International Conference on Data Mining (ICDM)* 841–850 (2020).
6. Deng, A. & Hooi, B. Graph neural network-based anomaly detection in multivariate time series. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35, 4027–4035 (2021).
7. Li, Z. et al. Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining* 3220–3230 (2021).
8. Xu, J., Wu, H., Wang, J. & Long, M. Anomaly transformer: Time series anomaly detection with association discrepancy. In *International Conference on Learning Representations* (2022).
9. Tuli, S., Casale, G. & Jennings, N. R. TranAD: Deep transformer networks for anomaly detection in multivariate time series data. *Proc. VLDB Endow.* **15**, 1201–1214 (2022).
10. Yang, Y., Zhang, C., Zhou, T., Wen, Q. & Sun, L. DCdetector: Dual attention contrastive representation learning for time series anomaly detection. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* 3033–3045 (2023).
11. Chen, P., Qi, Y., Zheng, P. & Hou, D. CauseInfer: Automatic and distributed performance diagnosis with hierarchical causality graph in large distributed systems. In *IEEE INFOCOM 2014–IEEE Conference on Computer Communications*, 1887–1895 (2014).
12. Kim, M., Sumbaly, R. & Shah, S. Root cause detection in a service-oriented architecture. In *Proceedings of the 2013 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems* 93–104 (2013).
13. Wang, P. CloudRanger: Root cause identification for cloud native systems. In *18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)* 492–502 (2018).
14. Meng, Y. Localizing failure root causes in a microservice through causality inference. In *IEEE/ACM 28th International Symposium on Quality of Service (IWQoS)* 1–10 (2020).
15. Li, Z. Practical root cause localization for microservice systems via trace analysis. In *IEEE/ACM 29th International Symposium on Quality of Service (IWQoS)* 1–10 (2021).
16. Yu, G. et al. Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 553–565 (2023).
17. Sundararajan, M., Taly, A. & Yan, Q. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, 3319–3328 (2017).
18. Lundberg, S. M. & Lee, S.-I. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, Vol. 30, 4765–4774 (2017).
19. Bento, J., Saleiro, P., Figueiredo, V. & Bizarro, P. TimeSHAP: Explaining recurrent models through sequence perturbations. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining* 2564–2573 (2021).
20. Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations* (2017).
21. Veličković, P. Graph attention networks. In *International Conference on Learning Representations* (2018).
22. Li, Y., Yu, R., Shahabi, C. & Liu, Y. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations* (2018).
23. Wu, Z., Pan, S., Long, G., Jiang, J. & Zhang, C. Graph WaveNet for deep spatial-temporal graph modeling. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence* 1907–1913 (2019).
24. Vaswani, A. Attention is all you need. In *Advances in Neural Information Processing Systems*, Vol. 30 (2017).
25. Wu, H., Xu, J., Wang, J. & Long, M. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems* Vol. 34, 22419–22430 (2021).
26. Nie, Y., Nguyen, N. H., Sinthong, P. & Kalagnanam, J. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations* (2023).
27. Kim, C., Mendiratta, V. B. & Thottan, M. Unsupervised anomaly detection and root cause analysis in mobile networks. In *2020 International Conference on COMmunication Systems & NETworkS (COMSNETS)* 176–183 (2020).
28. Jia, J. Transformer-based alarm context-vectorization representation for reliable alarm root cause identification in optical networks. In *European Conference on Optical Communication (ECOC)* 1–4 (2021).
29. Lantz, B., Heller, B. & McKeown, N. A network in a laptop: Rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks* 1–6 (2010).
30. Li, Z. Actionable and interpretable fault localization for recurring failures in online service systems. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* 996–1008 (2022).

Acknowledgements

The authors thank the anonymous reviewers and editors for their time and consideration. The authors also thank the colleagues who supported the construction and maintenance of the experimental network operation testbed used in this study.

Author contributions

D.L. conceived the study, developed the methodology, and drafted the manuscript. W.W. supervised the research, refined the study design, and revised the manuscript. X.L. collected and curated the data and participated in validation and analysis. W.Y. implemented the experiments, prepared the figures, and contributed to the interpretation of the results. K.H. contributed to the methodology, resources, and formal analysis. J.Z. contributed to the literature review, manuscript revision, and project administration. All authors reviewed the manuscript and approved the final version.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to W.W.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026